
Fast k -means Seeding Under The Manifold Hypothesis

Poojan Shah¹ Shashwat Agrawal¹ Ragesh Jaiswal¹

Abstract

We study *beyond worst case analysis* for the k -means problem where the goal is to model typical instances of k -means arising in practice. Existing theoretical approaches provide guarantees under certain assumptions on the optimal solutions to k -means, making them difficult to validate in practice. We adopt the *manifold hypothesis*, where data obtained in ambient dimension D concentrates around a low dimensional manifold of intrinsic dimension $d \ll D$, as a reasonable assumption to model real world clustering instances. We identify key geometric properties of datasets which have theoretically predictable scaling laws depending on the *quantization exponent* $\varepsilon = 2/d$ using techniques from optimum quantization theory. We show how to exploit these regularities to design a fast seeding method called Qkmeans¹ which provides $O(\varrho^{-2} \log k)$ approximate solutions to the k -means problem in time $O(nD \log k) + \tilde{O}(\varrho^{-1} k^{1+\gamma})$; where the exponent $\gamma = \varepsilon + \varrho$ for an input parameter $\varrho < 1$. This allows us to obtain new runtime - quality trade-offs. We perform a large scale empirical study² across various domains to validate our theoretical predictions and algorithm performance to bridge theory and practice for beyond worst case data clustering.

1. Introduction

Data clustering is a fundamental task in machine learning and data science. The goal is to partition a dataset into subsets so that points within the same subset (or cluster) are more similar to each other than to points in other clusters. Such partitions help uncover latent structure and compress

data into a small number of interpretable groups. Among the many approaches to clustering, the k -means formulation is perhaps the most widely used. In this formulation, we are given a dataset $\mathcal{X} = \{x_1, \dots, x_n\} \subset \mathbb{R}^D$, and our task is to find a set of k centers $C = \{c_1, \dots, c_k\} \subset \mathbb{R}^D$ which minimize the total within-cluster variance:

$$\text{cost}(\mathcal{X}, C) \stackrel{\text{def}}{=} \sum_{x \in \mathcal{X}} \min_{c \in C} \|x - c\|^2.$$

These centers induce a partition $\mathcal{X} = \bigcup_{j=1}^k C_j$, where C_j contains the points in the dataset $\mathcal{X}$ closest to c_j . An optimal set C_k^* achieves $\text{cost}(\mathcal{X}, C_k^*) = \text{opt}_k(\mathcal{X})$. Optimizing the k -means cost then serves as a proxy for uncovering some natural clustering of the dataset. This formulation has become a key component of algorithm design in several fields such as vector quantization and compression (Linde et al., 1980), approximate nearest neighbor search (Jégou et al., 2011a; Johnson et al., 2017) along with numerous others.

Given the immense importance of k -means in modern data processing, it is natural that the problem has been thoroughly investigated from both theoretical and empirical perspectives. The k -means problem is known to be NP-hard (Dasgupta, 2008; Mahajan et al., 2009), so it is unlikely to have a polynomial time algorithm. Much research has been done on designing polynomial time *approximation schemes*³ for the k -means problem. However, the algorithm most popularly used in practice⁴ to solve k -means instances is a heuristic known as the k -means algorithm (*not to be confused with the k -means problem*). This heuristic, also known as Lloyd’s iterations (Lloyd, 1982), iteratively improves the solution in several rounds. The heuristic starts with an arbitrarily chosen set of k centers. In every iteration, it (i) partitions the points based on the nearest center and (ii) updates the center set to the centroids of the k partitions. In the classical computational model, it is easy to see that every Lloyd iteration costs $O(nkD)$ time. This hill-climbing approach may get stuck at a local

¹Department of Computer Science and Engineering, Indian Institute of Technology Delhi. Correspondence to: Poojan Shah <shahpoojan2004@gmail.com>.

Proceedings of the 43rd International Conference on Machine Learning, Seoul, South Korea. PMLR 306, 2026. Copyright 2026 by the author(s).

¹The Q stands for Quantization, as the analysis of our algorithm was motivated by results from optimum quantization theory.

²Our code is available at [This Repository](#)

³These were developed in a long line of research by (Kumar et al., 2010; Jaiswal et al., 2014; 2015; Cohen-Addad, 2018; Frigstad et al., 2019; Cohen-Addad et al., 2019; Bhattacharya et al., 2020)

⁴Ranked as the second most popular data mining algorithm in (Wu et al., 2008).

minimum or take a huge amount of time to converge, and hence, it does not give provable guarantees on the quality of the final solution or the running time (Dasgupta, 2003; Har-Peled & Sadri, 2005; Arthur & Vassilvitskii, 2006a;b). In practice, Lloyd’s iterations are usually preceded by the k -means++ algorithm (Arthur & Vassilvitskii, 2007), a fast sampling-based approach for picking the initial k centers that also gives an approximation guarantee. So, Lloyd’s iterations, preceded by the k -means++ algorithm, give the best of both worlds, theory and practice. Hence, it is unsurprising that a lot of work has been done on these two algorithms. The work ranges from efficiency improvements in specific settings to implementations in distributed and parallel models.

Beyond worst case analysis. A large fraction of theoretical algorithms for k -means are designed to be used as *off the shelf* algorithms in a *data oblivious* manner. This means that they assume that the input dataset $\mathcal{X}$ can be any arbitrary set of points in $\mathbb{R}^D$ and provide worst case guarantees. The pervasive issue is that algorithms designed to resist pathological instances, i.e. inputs that maximize resource usage-often yield complexity bounds far removed from their typical empirical performance. The realization that real datasets rarely resemble these theoretical worst-case examples provides the core motivation for moving beyond pessimistic, data-oblivious constraints (Roughgarden, 2019; Balcan, 2021; Agarwal et al., 2015). This *data driven* paradigm acts as a necessary bridge between theoretical analysis and practical applicability and so has gained attention in the algorithm design community.

For k -means in particular, this comprises of several approaches. The performance of Lloyd-type methods and k -means++ under separability conditions was analyzed by (Ostrovsky et al., 2013; Jaiswal & Garg, 2012). An instance $(\mathcal{X}, k)$ of the k -means problem is said to be α -separated if $\frac{\text{opt}_k(\mathcal{X})}{\text{opt}_{k-1}(\mathcal{X})} \leq \alpha$. This definition is motivated by the usual practice of using the *elbow method* to determine a good value of k to be used for clustering $\mathcal{X}$. Another line of research studies the notion of *approximation stability* in (Balcan et al., 2009; Blum, 2021; Agarwal et al., 2015). Approximation stability is the idea that any clustering whose cost is close to optimal must be similar to the ground truth clustering. An instance $(\mathcal{X}, k)$ is called (c, α) -approximation stable with respect to the cost function cost and a ground truth target clustering C_T if every c approximate solution to cost is atmost α far from C_T in the sense of fraction of points misclustered. Other proposals include distributional stability (Bilu & Linial, 2010), spectral stability (Kumar & Kannan, 2010) and perturbation resilience (Awasthi et al., 2010). (Cohen-Addad & Schwiegelshohn, 2017) studied the structural consequences of these notions. A common theme

across these proposals is assuming some properties of the optimal k -means clustering, which makes an empirical study of these notions quite difficult. Indeed, it is a non trivial problem to figure out how to check if a given dataset instance follows these assumptions. Although these notions have been studied theoretically, not much work has been done in studying whether they hold (even approximately) true for real world datasets, in part due to the fact that these assumptions bestow constraints on the optimal cost, which are computationally hard to verify.

Motivating Questions The above discussion naturally leads to the following questions:

1. Can we identify structural properties of real-world datasets that are easily verifiable, well motivated in theory and empirically hold true for diverse modern datasets ?
2. If so, can we exploit the existence of such structural properties to design better algorithms for k -means clustering?

Contributions. In order to tackle these questions, we present the following contributions, which are discussed in detail in the next section :

1. We adopt the manifold hypothesis - a well-established and empirically validated model for high-dimensional data - as a structural assumption for beyond-worst-case analysis of k -means.
2. Under this framework, we identify geometrical parameters which have predictable scaling law behaviour. We use this structure to design a fast seeding procedure through analyzing a *perturbed* version of k -means++ sampling.
3. We perform a large scale empirical study to validate our assumptions and the performance of our seeding algorithm.

1.1. Our Contributions

We propose the idea that the *manifold hypothesis* is a good candidate to derive such structural properties. The manifold hypothesis states that although real-world data often reside in high-dimensional ambient spaces, their probability mass tends to concentrate near low-dimensional, smooth submanifolds $\mathcal{M} \subset \mathbb{R}^D$ with intrinsic dimension $d \ll D$. This hypothesis forms the conceptual bedrock for explaining the success of modern deep-learning systems (Bengio et al., 2013) and has subsequently attracted thorough theoretical and experimental investigation (Fefferman et al., 2016; Brown et al., 2023; Bordt et al., 2023) and hence is widely accepted and established. We formally state this:

Assumption 1. (*Manifold hypothesis assumption*) The dataset $\mathcal{X} = \{x_1, \dots, x_n\} \subset \mathbb{R}^D$ is generated by drawing n independent and identically distributed samples from a density function $f : \mathcal{M} \rightarrow \mathbb{R}_+$ whose support $\text{supp}(f)$ is a measurable subset of a smooth, compact d -dimensional submanifold $\mathcal{M} \subset \mathbb{R}^D$.

From the point of view of beyond-worst-case analysis, the manifold hypothesis offers a different type of structural regularity: instead of assuming properties about the *optimal solution*, it assumes geometric regularity of the *input distribution*. This makes it attractive for algorithm design, as the assumption is intrinsic to the data rather than to the algorithm’s output.

Moreover, it is compatible with the increasing body of empirical evidence that high-dimensional machine learning data has low intrinsic dimension (Tempeczyk et al., 2022; Horvat & Pfister, 2022). Finally, from a practical standpoint, the manifold hypothesis is more aligned with modern applications than separation- or stability-based assumptions. While the latter are tailored to idealized clustering scenarios, the manifold hypothesis is already the implicit structural model underlying essentially all deep learning architectures used in vision, NLP, physics, and generative tasks (Bengio et al., 2013; Loaiza-Ganem et al., 2024).

What does this imply for the k -means problem? A fundamental question is to study how the optimal k means cost decreases with the number of clusters k . This question was explored for Euclidean spaces in an early work of (Zador, 1982), who showed asymptotic laws for the optimal *quantization cost* given by $\Delta_k(f) = \inf_{|C|=k} \Delta(f, C)$ where $\Delta(f, C) = \int_{\mathcal{M}} \min_{c \in C} \|x - c\|^2 f(x) dx$. This led to the development of the field now known as *optimum quantization theory*, for a detailed exposition of which we point the readers to (Gray & Neuhoﬀ, 2006; Graf & Luschgy, 2000). Zador’s results were generalized to manifolds by (Gruber, 2004), which are of interest to us :

Fact 2. (Gruber, 2004) Suppose f is a probability density according to Assumption 1 and $S_k = \{s_1, \dots, s_k\} \subset \mathbb{R}^D$ be an optimal k -quantizer of f i.e., $S_k \in \arg \min_{|C|=k} \Delta(f, C)$.

Define the parameters $\beta_k(f)$ and $\eta_k(f)$ as follows :

$$\beta_k(f) \stackrel{\text{def}}{=} \frac{\Delta_1(f)}{\Delta_k(f)} \quad \eta_k(f) \stackrel{\text{def}}{=} \frac{\max_{i \neq j \in [k]} \|s_i - s_j\|}{\min_{i \neq j \in [k]} \|s_i - s_j\|}$$

Then, the following holds :

$$\beta_k(f) \in O_f(1) \cdot k^\varepsilon \quad \eta_k(f) \in O_f(1) \cdot k^{\varepsilon/2}$$

where $\varepsilon = 2/d$ is referred to as the *quantization exponent* and $O_f(\cdot)$ hides constants depending only on f .

Remark 3. We mention that such scaling laws are prevalent in several research areas. Notable examples include the

scaling law between word frequency and word rank in large text documents, referred to as Zipf’s law (Zipf, 1949) and the power law distribution of node degrees in large scale networks (Barabasi & Albert, 1999). These are frequently used to guide algorithm design and algorithm analysis.

We now define these parameters for finite datasets :

Definition 4. For a dataset $\mathcal{X} = \{x_1, \dots, x_n\}$ and a set of centers $C = \{c_1, \dots, c_k\}$ we define the following:

$$\beta(\mathcal{X}, C) := \frac{\text{cost}(\mathcal{X}, \mu)}{\text{cost}(\mathcal{X}, C)} \quad \beta_k(\mathcal{X}) := \frac{\text{opt}_1(\mathcal{X})}{\text{opt}_k(\mathcal{X})}$$

$$\eta(\mathcal{X}, C) := \frac{\max_{i \neq j} \|c_i - c_j\|}{\min_{i \neq j} \|c_i - c_j\|} \quad \eta(\mathcal{X}) := \frac{\max_{i \neq j} \|x_i - x_j\|}{\min_{i \neq j} \|x_i - x_j\|}$$

Here, $\mu = \mu(\mathcal{X})$ is the centroid of the dataset $\mathcal{X}$.

Note that we always have $\beta(\mathcal{X}, C) \leq \beta_k(\mathcal{X})$ and $\eta(\mathcal{X}, C) \leq \eta(\mathcal{X})$ for all $C = \{c_1, \dots, c_k\}$, as long as the centers belong to the dataset itself i.e., $C \subset \mathcal{X}$. We show the following finite sample bounds :

Theorem 5. (Scaling laws) Suppose $\mathcal{X}$ is sampled according to Assumption 1 then the following hold with probability at least $1 - \frac{1}{\text{poly}(n)}$:

$$\beta_k(\mathcal{X}) \in \left(1 + O_f\left(\sqrt{\frac{kD \log n}{n}}\right)\right) \cdot k^\varepsilon$$

$$\eta(\mathcal{X}) \in O_f(1) \cdot n^{3\varepsilon/2}$$

Next, we design a fast rejection-sampling based seeding scheme to accelerate k -means++ with novel runtime-approximation tradeoffs. Our algorithms are designed in a way which allows us to exploit the scaling laws. This allows us to improve upon existing approaches aimed at speeding up k -means++ under the manifold hypothesis. We formally state our main results as follows :

Theorem 6. (Main Theorem) Let $\mathcal{X} = \{x_1, \dots, x_n\} \subset \mathbb{R}^D$ be a dataset. Let $\beta_k(\mathcal{X})$ and $\eta(\mathcal{X})$ be the data dependent parameters introduced in Definition 4. Then, there exists an algorithm which additionally takes as input parameters $\varrho < 1, m \in \mathbb{N}$ and outputs a set C of k centers such that

$$\mathbb{E}[\text{cost}(\mathcal{X}, C)] \in O(\varrho^{-2}) \cdot \left(\underbrace{\log k \cdot \text{opt}_k(\mathcal{X})}_{k\text{-means++ guarantee}} + \underbrace{k^{-\varrho m / \beta_k(\mathcal{X})} \cdot \text{opt}_1(\mathcal{X})}_{\text{exponentially decaying additive term}} \right)$$

using a one-time preprocessing time of $O(nD \log k)$, with each seeding taking time $\tilde{O}(m(k \log \eta(\mathcal{X}))^{1+\varepsilon} \log \beta_k(\mathcal{X}))$.

The above theorem holds true for *all* datasets without any assumptions and hence the bounds are in terms of the data dependent parameters $\beta = \beta_k(\mathcal{X})$ and $\eta = \eta(\mathcal{X})$. We can see that for fixed η, β , the runtime consists of an $O(nD \log k)$

term, which is required just for reading the input $\mathcal{X}$, with a $\log k$ overhead for dimensionality reduction as part of the preprocessing, and an additional term which is *almost* linear in k . The approximation guarantee consists of the standard $O(\log k)$ guarantee along with an additive, scale invariant and exponentially decaying term depending on the variance of the dataset. Due to the strong decaying of the additive term, it can be removed as follows :

Corollary 7. (*Rejection sampling for k -means++*) Let $\mathcal{X} = \{x_1, \dots, x_n\} \subset \mathbb{R}^D$ be a dataset. Then there exists an algorithm which additionally takes as input a parameter $\varrho < 1$ and outputs a set C of k centers such that

$$\mathbb{E}[\text{cost}(\mathcal{X}, C)] \in O(\varrho^{-2} \log k) \text{opt}_k(\mathcal{X})$$

using a one-time preprocessing time of $O(nD \log k)$, with each seeding taking expected time $\tilde{O}(\varrho^{-1} \beta_k(\mathcal{X})(k \log \eta(\mathcal{X}))^{1+\varepsilon} \log \beta_k(\mathcal{X}))$, and $\tilde{O}(\cdot)$ hides $\log k, \log n$ factors

Finally, we can combine the scaling laws from Theorem 5 along with Theorem 6 to obtain the following result for k -means clustering under the manifold hypothesis:

Corollary 8. (*Rejection sampling under manifold hypothesis*) Let $\mathcal{X} = \{x_1, \dots, x_n\} \subset \mathbb{R}^D$ be a dataset generated according to Assumption 1. Then there exists an algorithm which additionally takes as input a parameter $\varrho < 1$ and outputs a set C of k centers such that

$$\mathbb{E}[\text{cost}(\mathcal{X}, C)] \in O(\varrho^{-2} \log k) \text{opt}_k(\mathcal{X})$$

using a one-time preprocessing time of $O(nD \log k)$, with each seeding taking expected time $\tilde{O}(\varrho^{-1} k^{1+\gamma})$, where $\gamma = \varepsilon + \varrho$, and $\tilde{O}(\cdot)$ hides $\log k, \log n$ factors

Remark 9 (Estimating the data-dependent parameters). The parameters $\beta_k(\mathcal{X})$ and $\eta(\mathcal{X})$ appearing in Theorem 6 and Corollaries 7-8 are in general, not known before the execution of the algorithm. We address this issue by discussing how to estimate them prior to execution.

1. The parameters can be upper bounded by the dataset aspect ratio $\eta(\mathcal{X})$, $\beta_k(\mathcal{X}) \leq \eta(\mathcal{X})^2$. The aspect ratio can be approximated in $O(n \log n)$ time (Shamos & Hoey, 1975), providing worst-case bounds that hold for any dataset without any assumptions. However, our experiments show that these are often too pessimistic.
2. Under Assumption 1, the parameters exhibit predictable scaling behavior. In particular, one can estimate the intrinsic dimension d using the maximum likelihood estimator of (Levina & Bickel, 2005), and obtain an estimate of the quantization exponent $\varepsilon = 2/d$. This yields $\beta_k(\mathcal{X}) \lesssim k^{2/d}$, which is significantly tighter and more informative than the worst-case bound. As shown in Section 3, these estimates are stable across datasets and domains.

3. A practical alternative is to compute k -means costs for a small set of values $k \in O(1)$, fit the scaling laws, and extrapolate to larger k . Our experiments show that the resulting fits are highly consistent (typically $R^2 \geq 0.95$).

Finally, given any estimate of $\beta_k(\mathcal{X})$ obtained via the above approaches, one can set $m = O(\beta_k(\mathcal{X}))$ to ensure a negligible contribution from the additive term in Theorem 6.

Empirical results. We perform extensive empirical analysis to test whether our theoretical assumptions are reasonable for practical datasets. Our study captures cross-domain datasets covering image data, image and text embeddings, as well as tabular data captured from sensors and scientific experiments. Our assumptions can be easily checked by computing the cost for various values of k , then evaluating the geometric properties of the resulting clusters. We find that the scaling laws strongly manifest in these datasets. We also analyze the effect of intrinsic dimension on these laws using independent ID estimates. Finally, we observe that our seeding method gives about an order of magnitude speedup for k -means++ seeding on high dimensional datasets when compared to prior work.

1.2. Comparison with prior work

Speeding up k -means++. Let us discuss the methods proposed to accelerate k -means++ so as to position our results in the literature. Directly implementing k -means++ provides an $O(\log k)$ approximation guarantee in $O(nkD)$ time, which quickly becomes prohibitively slow even for moderate sized datasets. The problem is that k -means++ is inherently a sequential procedure where the new center chosen depends on the previously chosen centers. k -means++ has also been extended to distributed settings (Bahmani et al., 2012; Bachem et al., 2017a; 2018), but our results are for clustering on a single node.

MCMC methods. In the line of work (Bachem et al., 2016b;a), the authors propose to approximate the k -means++ distribution using an appropriately designed markov chain, whose limiting distribution is equal to the D^2 distribution. Both of these algorithms have runtime of $\Omega(k^2)$, thus making them slow for use-cases when a dataset is required to be partitioned into thousands of clusters⁵. In comparison, Qkmeans does not suffer from this $\Omega(k^2)$ dependence, but has $O(k^{1+\gamma})$ dependence where the exponent $\gamma := \varepsilon + \varrho$. As we shall see, $\varepsilon \sim 0.1 - 0.2$ in real large scale datasets. Moreover, the MCMC methods often have an additive term which decays as $O(1/m)$ where m is the markov chain length, while we obtain an exponential $O(k^{-\varrho m/\beta})$ decay in our results.

⁵This was empirically demonstrated in Section 6 of (Cohen-Addad et al., 2020a).

ANNS techniques. In (Cohen-Addad et al., 2020a), the authors provide a speedup for k -means++ using a variety of algorithmic techniques which include hierarchical tree embeddings, approximate nearest neighbor search and rejection sampling. However, constructing this embedding brings up dependence on the aspect ratio of the dataset Δ , which in general may be unbounded in terms of the input size and requires special pre-processing. They then use rejection sampling to convert samples from the D^2 distribution on the tree metric to a sample in the euclidean metric, which introduces an additional factor of $O(D^2)$. In comparison, our algorithm does not require such metric embeddings and instead depends on simply sampling from an ℓ_2 -norm distribution which can be computed in a lightweight pre-processing step. Moreover, we have better dependence on the ambient dimension $O(D)$ vs $O(D^3)$ and prove improved approximation guarantee $O(\varrho^{-2})$ vs $O(\varrho^{-3})$. We also mention several works which try to speed up k -means++ or Lloyd’s iterations by maintaining some nearest neighbor data structure (Pelleg & Moore, 1999; Kanningo et al., 2002; Elkan, 2003; Hamerly, 2010; Wang et al., 2012; Ding et al., 2015; Bottesch et al., 2016; Newling & Fleuret, 2016; Curtin, 2017), but do not provide any theoretical guarantee on the solution quality or runtime for their proposed methods.

Dimensionality reduction. In (Charikar et al., 2023), the authors propose to project the dataset onto a randomly sampled unit vector and then perform k -means++ in 1D in $O(n \log n)$ time. They show that when this 1D solution is lifted back to D dimensions, it provides an $O(k^4 \log k)$ approximation guarantee. Although this is extremely fast, they report clustering costs $10\text{-}100\times$ larger than those provided by k -means++. They also propose a coresets-based method which theoretically runs in $O(nD + n \log n) + O(k^5 D)$ time to provide $O(\log k)$ approximation. In comparison, our method has a much lower dependence on k . There is a line of research (see (Makarychev et al., 2019) and the references therein) on dimensionality reduction for k -means which show that the ambient dimension can be reduced to $O(\log k)$ while preserving the k -means cost.

Coresets. Coresets are a way to compress the dataset by computing a weighted subset of the data. We refer the reader to (Bachem et al., 2017b; Feldman, 2020) for becoming familiar with coresets, which have now become ubiquitous tools for k -means clustering. This long line of research culminated in (Draganov et al., 2024), where a coreset construction in near-linear time $\tilde{O}(nD \log \log \Delta)$ of size $O(k/\varepsilon^4)$ is proposed which can then be used in conjunction with standard k -means++ to obtain an $O(nD \log \log \Delta + k^2 D)$ algorithm having $O(\log k)$ guarantee. We conclude by providing a summary of this discussion in Table 1. We mention that our results provide new theoretical tradeoffs as well as insights into the structure of input data for k -means cluster-

Algorithm 1 PREPROCESS($\mathcal{X}$)

1. For $\varepsilon_{\text{JL}} \in (0, 1/4)$, apply the JL-like dimensionality reduction from (Makarychev et al., 2019) in $O(nD \log k)$ time to obtain $D \in O(\log(k/\varepsilon_{\text{JL}})/\varepsilon_{\text{JL}}^2)$
 2. Compute the mean $\mu(\mathcal{X}) := \frac{1}{n} \sum_i x_i$ of the dataset
 3. Center each $x_i \leftarrow x_i - \mu(\mathcal{X})$, $1 \leq i \leq n$
 4. Compute $\|\mathcal{X}\|_{\text{F}}^2 := \sum_i \|x_i\|^2$
 5. Compute $\kappa(x_i) \leftarrow \|x_i\|^2 / \|\mathcal{X}\|_{\text{F}}^2$, $1 \leq i \leq n$
-

Algorithm 2 SAMPLE($\mathcal{L}, C, m$)

1. Initialize $iter \leftarrow 0$ and sample $s \sim \text{UNIFORM}(\mathcal{X})$
 2. **while** $iter < m \ln k$ **do**
 - $iter \leftarrow iter + 1$
 - Sample $x \sim \kappa(\cdot | C)$, where $\kappa(x | C) := \frac{\|x\|^2 + \|c_1\|^2}{\|\mathcal{X}\|_{\text{F}}^2 + n\|c_1\|^2}$
 - Sample $R \sim \text{UNIFORM}(0, 1)$
 - Compute $r \leftarrow \frac{\text{cost}(x, \text{Query}(\mathcal{L}, x))}{2\varrho^{-1}(\|x\|^2 + \|c_1\|^2)}$
 - **if** $R \leq r$ **then** $s \leftarrow x$ and **break**
 3. **Return** s
-

ing, along with providing a method to exploit them.

2. Seeding Algorithm

At a high level, our algorithm works by *simulating* the D^2 sampling step of k -means++ using rejection sampling. The idea is simple: instead of sampling directly from the D^2 distribution - which is expensive to update after every new center - we sample from a much easier distribution and correct the bias using a rejection step. The efficiency of this correction is controlled by a single quantity: the max Rényi divergence between the two distributions.

Suppose μ is a target distribution over a finite domain $\mathcal{X}$, and ν is a proposal distribution from which sampling is easy. Rejection sampling allows us to draw exact samples from μ using only samples from ν , provided that μ is not too “peaked” relative to ν .

Definition 10. Let μ and ν be probability distributions supported on $\mathcal{X}$. The max Rényi divergence of μ from ν is defined as

$$D_{\infty}(\mu \| \nu) \stackrel{\text{def}}{=} \sup_{x \in \mathcal{X}} \log \frac{\mu(x)}{\nu(x)}.$$

Intuitively, $\exp(D_{\infty}(\mu \| \nu))$ measures the worst-case mismatch between μ and ν , and directly determines how many proposal samples we need before one is accepted.

Lemma 11 (Rejection sampling guarantee). Let μ and ν be distributions over a finite set $\mathcal{X}$ with $D_{\infty}(\mu \| \nu) < \infty$, and

Table 1. Comparison of time complexity and approximation guarantees for accelerating k -means++ seeding. For our method, we empirically show $\beta \sim k^\varepsilon$ and $\log \eta \sim \varepsilon \log k$, where ε is the quantization exponent. In the table, Δ is the aspect ratio of the dataset and β, η are the parameters from Theorem 5. Note that the relation $\eta \leq \Delta$ always holds true. Moreover, under the manifold hypothesis the overall exponent γ is defined as $\gamma = \varepsilon + \varrho$.

APPROACH	TIME COMPLEXITY	APPROXIMATION GUARANTEE
<i>Prior work</i>		
(Bachem et al., 2016a)	$O(nD) + O(\varepsilon^{-1} k^2 D \log k \varepsilon^{-1})$	$O(\log k) \text{opt}_k + \varepsilon \text{opt}_1$
(Bachem et al., 2018)	$O(nD) + O(\varepsilon^{-2} k^2 D^2 \log k)$	$O(\log k) \text{opt}_k + \varepsilon \text{opt}_1$
(Cohen-Addad et al., 2020a)	$O(n(D + \log n) \log(\Delta D))$ $+ O(\varrho^{-1} k D^3 \log \Delta (n \log \Delta)^{O(\varrho)})$	$O(\varrho^{-3} \log k) \text{opt}_k$
(Charikar et al., 2023)	$O(nD) + O(n \log n)$	$O(k^4 \log k) \text{opt}_k$
(Charikar et al., 2023)	$O(nD) + O(n \log n)$ $+ O(\varepsilon^{-2} k^5 D \log^2 k)$	$O(\log k) \text{opt}_k$
(Draganov et al., 2024)	$O(nD \log \log \eta) + O(\varepsilon^{-4} k^2 D)$	$O(\log k) \text{opt}_k$
(Shah & Jaiswal, 2025)	$O(nD) + O(\Delta^2 k^2 D)$	$O(\log k) \text{opt}_k$
<i>Our results</i>		
Theorem 6	$O(nD) + O(mD \log k (k \log \eta)^{1+\varepsilon})$	$O(\varrho^{-2}) (\log k \text{opt}_k + k^{-\Omega(m/\beta)} \text{opt}_1)$
Corollary 7	$O(nD) + O(\varrho^{-1} \beta D (k \log \eta)^{1+\varepsilon})$	$O(\varrho^{-2} \log k) \text{opt}_k$
Corollary 8	$O(nD) + O(\varrho^{-1} (k \log n)^{1+\gamma} D)$	$O(\varrho^{-2} \log k) \text{opt}_k$

Algorithm 3 QKMEANS($\mathcal{X}, k, m$)

1. PREPROCESS($\mathcal{X}$)
2. Initialize the data structure INIT($\mathcal{L}, \varrho$)
3. Sample $c_1 \sim \text{UNIFORM}(\mathcal{X})$
4. Set $C \leftarrow \{c_1\}$, insert c_1 into $\mathcal{L}$, and set $C_1 \leftarrow \{c_1\}$
5. **for** $t = 2$ **to** k **do**
 - Sample $c_t \leftarrow \text{SAMPLE}(\mathcal{L}, C, m)$
 - Insert c_t into $\mathcal{L}$
 - Update $C_t \leftarrow C_{t-1} \cup \{c_t\}$
6. **Return** C_k

set $M = \exp(D_\infty(\mu \parallel \nu))$. Then:

1. Conditioned on acceptance, Algorithm 4 outputs an exact sample from μ .
2. Each iteration succeeds with probability at least $1/M$.
3. Running the algorithm for $M \ln k$ iterations succeeds with probability at least $1 - 1/k$.

Proof. In one iteration, the probability of accepting a point x is $\nu(x) \cdot \mu(x) / (M \nu(x)) = \mu(x)/M$. Thus, conditioned on acceptance, the output distribution is exactly μ . The total acceptance probability is $1/M$, and repeating for $M \ln k$ iterations fails with probability at most $(1 - 1/M)^{M \ln k} \leq 1/k$. $\square$

For performing k -means++ seeding, we are interested in sampling from the D^2 distribution given by $\pi(x|C) =$

Algorithm 4 REJECTSAMPLE(μ, ν, M)

1. **for** $i = 1$ **to** $M \ln k$ **do**
 - (a) Sample $x \sim \nu$
 - (b) Sample $R \sim \text{UNIFORM}(0, 1)$
 - (c) **if** $R \leq \frac{\mu(x)}{M \nu(x)}$ **then**
 - i. **return** x
2. **Fail**

$\text{cost}(x, C) / \text{cost}(\mathcal{X}, C)$ for each $x \in \mathcal{X}$, which is our target. What remains to be done is to select a suitable proposal distribution, which we now define as follows :

Definition 12. Let c_1 be the first center which is chosen uniformly at random from $\mathcal{X}$ by Algorithm 3. The distribution $\kappa(\cdot|C)$ is defined as follows :

$$\kappa(x|C) = \frac{\|x\|^2 + \|c_1\|^2}{\|\mathcal{X}\|_F^2 + n\|c_1\|^2}$$

Remark 13. The motivation behind selecting this particular distribution comes from a line of work in the theoretical computer science community called “dequantized quantum machine learning” which often uses similar ℓ^2 norm distributions over the data. More recently, this distribution was introduced in the context of clustering algorithms by works such as (Cornelissen et al., 2025; Shah & Jaiswal, 2025).

Using the Cauchy-Schwarz inequality, we can provide an upper bound on the max-Rényi divergence as follows :

Lemma 14. Let $\pi(\cdot|C)$ be the target D^2 distribution and $\kappa(\cdot|C)$ be the proposal distribution defined in Definition 12. Then the following holds :

$$\mathbb{E}[D_\infty(\pi(\cdot|C)\|\kappa(\cdot|C))] \leq \log\left(4 \cdot \frac{\text{opt}_1(\mathcal{X})}{\text{opt}_k(\mathcal{X})}\right)$$

Notice that the expression inside the $\log(\cdot)$ is exactly the $\beta_k(\mathcal{X})$ parameter which we introduced earlier. Under Assumption 1 we can use Theorem 5 to obtain the following:

Lemma 15. Suppose the dataset $\mathcal{X}$ is generated according to Assumption 1, then the following holds :

$$\mathbb{E}[D_\infty(\pi(\cdot|C)\|\kappa(\cdot|C))] \in O\left(\frac{\log k}{d}\right)$$

Thus we can perform k -means seeding by first sampling a uniformly random point as a centre, and subsequently calling Algorithm 4 $k-1$ times with μ, ν being the distributions $\pi(\cdot|C), \kappa(\cdot|C)$ respectively, where C is the current set of centers. In order to employ the rejection sampling idea, the probability of accepting a sampled point $x \in \mathcal{X}$ needs to be proportional to $\text{cost}(x, C)^2$. However, computing this quantity takes $\Theta(kD)$ time, which contributes $\Theta(k^2D)$ time on performing this sampling k times. To mitigate this quadratic dependence on k , we apply another optimization technique; we use an approximate nearest neighbors (ANNS) data structure to approximate the distance between x and the closest center. We use one such ANNS data structure developed by (Cohen-Addad et al., 2020a) which is based on locality sensitive hashing (details in Appendix C.2). This data structure is specified by the following properties:

Lemma 16. (LSH-based ANNS data structure) For any set of points $P = \{p_1, \dots, p_t\} \subset \mathbb{R}^D$ and parameter $\varrho < 1$, there exists a data structure $\mathcal{L}$ with operations Insert and Query such that with probability at least $1 - 1/t$, the following guarantees hold ⁶

1. Insert($\mathcal{L}, p$) inserts the point $p \in P$ into $\mathcal{L}$ in time $O(D \log \eta(t \log \eta)^\varrho)$.
2. Query($\mathcal{L}, p$) returns a point such that $\|p - \text{Query}(\mathcal{L}, p)\| \leq \frac{1}{\sqrt{\varrho}} \cdot \min_{q \in \mathcal{L}} \|p - q\|$ and the query time is $O(D \log \eta(t \log \eta)^\varrho)$.
3. $\mathcal{L}$ is monotone under insertions: the distance between p and Query($\mathcal{L}, p$) is non-increasing after inserting more points.

Here, η is the aspect ratio of the set P defined as $\eta(P) = \frac{\max_{x \neq y \in P} \|x - y\|}{\min_{x \neq y \in P} \|x - y\|}$.

We take one more step. We are passing a parameter $m \in \mathbb{N}$ in our input that is used to give a threshold on the time complexity of our seeding algorithm. We use this parameter to control the number of iterations in Rejection Sampling; if

we do not output a sample after $m \ln k$ iterations of sampling from $\kappa(\cdot|C)$, we terminate and instead output a sample from the uniform distribution on $\mathcal{X}$. With these add-ons, our idea for D^2 sampling via rejection sampling is given in Algorithm 2.

It is important to observe that both these add-ons to our original rejection sampling strategy of Algorithm 4 distort the output distribution from D^2 . As a consequence, we no longer have the $O(\log k)$ approximation guarantee readily available to us. We instead have an approximation guarantee which is dependent on m and ϱ as a part of Theorem 6. We give a detailed proof of the new approximation guarantee in Appendix C.3; to model the new distribution of cluster centers, we analyze an abstract version of k -means++ which we call (ϱ, δ) - k -means++, where instead of sampling from the standard D^2 distribution $\pi(\cdot|C)$, we instead have to sample from a perturbed distribution defined as follows .

Definition 17 ((ϱ, δ) perturbed D^2 distribution). Let $\mathcal{X} = \{x_1, \dots, x_n\} \subset \mathbb{R}^d$ be a dataset and $S \subset X$ be a set of centers sampled from X . Suppose that the data structure $\mathcal{L}$ with parameter $\varrho < 1$ is successful and each $c \in S$ is inserted into the data structure $\mathcal{L}$. Then for each $x \in \mathcal{X}$, we define the (ϱ, δ) perturbed D^2 distribution as :

$$\pi_{(\varrho, \delta)}(x|S) = (1 - \delta) \cdot \pi^{\mathcal{L}}(x|S) + \delta \cdot \frac{1}{n}$$

Our technical contribution is to analyze the seeding quality of our algorithm under this distribution. This requires a careful potential based analysis to control the effect of the parameters (ϱ, δ) without introducing any large constant factors. We emphasize that this is a non trivial task since it is not known whether perturbing the D^2 distribution by a small amount translates to a similar guarantee in the solution cost or not (Grunau et al., 2023). We leverage the specific structure of the perturbation to show the following.

Theorem 18 (Approximation guarantee for (ϱ, δ) - k -means++). Let $\mathcal{X} \subset \mathbb{R}^D$ be any dataset which is to be partitioned into k clusters. Let C be the set of centers returned by Algorithm 6 for any $\delta \in (0, 0.5)$ and $\varrho < 1$. The expected cost $\mathbb{E}[\text{cost}(\mathcal{X}, C)]$ equals:

$$O(\varrho^{-2} \log k) \text{opt}_k(\mathcal{X}) + \delta \cdot O(k + \varrho^{-2} \log k) \text{opt}_1(\mathcal{X})$$

If we let $m = \infty$, we can analyze the expected time of our algorithm and the approximation guarantee only depends on ϱ , which is given in Corollary 7. Finally, by plugging in the manifold hypothesis assumption, we obtain Corollary 8.

3. Empirical Study

In this section, we empirically evaluate the extent to which the theoretical predictions developed in this paper are reflected in real-world data. The complete details are in Appendix D.

Compute environment. All experiments were conducted on a dual-socket machine equipped with 2.20 GHz

DATASET	n	d
MNIST	60,000	784
FMNIST	60,000	784
CIFAR10	50,000	3,072
CIFAR100	50,000	3,072
MNIST (CLIP)	60,000	512
FMNIST (CLIP)	60,000	512
CIFAR10 (CLIP)	50,000	512
CIFAR100 (CLIP)	50,000	512
REDDIT	100,000	384
STACKEXCHANGE	20,000	384
SUSY	5,000,000	18
HAR	10,299	561

Table 2. Datasets used in experiments with sample size n and ambient dimension d .

IntelXeonGold5220R running Linux on x86-64 architecture.

Datasets. We include popular image datasets, both as raw pixels as well as image embeddings which are produced through joint vision-language models such as CLIP-ViT-B/32 (Radford et al., 2021); text datasets in the form of text embeddings generated by the All-MiniLM-L6-v2 model (Wang et al., 2020); tabular datasets which record sensor data as well as data generated by scientific experiments. The details of the datasets used are presented in Table 2.

Scaling Laws. We test the hypothesis that the data-dependent parameters $\beta = \beta(\mathcal{X}, C)$ and $\eta = \eta(\mathcal{X}, C)$ grow polynomially with the number of clusters k . For each dataset, we compute k -means clusterings for $k \in \{5, 10, 50, 100, 250, 500, 750, 1000\}$ using 10 independent runs of standard k -means++ followed by Lloyd’s iterations. We fit linear regressions to $\log \beta$ and $\log \eta$ versus $\log k$ and report slopes, R^2 , and 95% confidence intervals. Figure 1 shows that across datasets and modalities, both quantities exhibit reasonable linear behavior in log-log coordinates. It is important to note that although we are not able to provide bounds on η in term of k , the experiments show that $\log \eta$ scales atmost linearly with $\log k$. The estimated quantization exponent $\hat{\epsilon}$ is the slope of the best fit line for the $\log \beta$ vs $\log k$ plot.

Intrinsic dimension. We test the hypothesis that the empirically estimated quantization exponent $\hat{\epsilon}$ is inversely proportional to the intrinsic dimension. Independent intrinsic dimension estimates $\hat{d}_{MLE}$ are obtained by using the popular maximum likelihood method (MLE) (Levina & Bickel, 2005). For a dataset $\mathcal{X} = \{x_1, \dots, x_n\}$, let $T_j(x_i)$ denote the distance of the j th nearest neighbor of x_i from x_i . For a

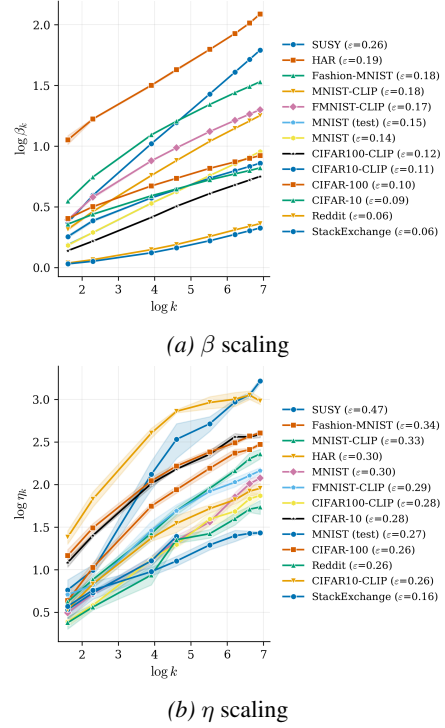


Figure 1. Scaling behavior across datasets

hyperparameter k , the ID estimate is calculated as :

$$\hat{d}_{MLE}(\mathcal{X}, k) := \frac{1}{n} \sum_{i=1}^n \left(\frac{1}{k-1} \sum_{j=1}^{k-1} \log \frac{T_k(x_i)}{T_j(x_i)} \right)^{-1}$$

For each dataset we take 10 independent samples each of size $n' = 10,000$ to estimate the ID for a particular value of k . This is repeated for $k \in \{5, 10, 20, 50, 100\}$ and the average is reported. We plot the estimates $\hat{\epsilon}$ of the quantization exponent against $\hat{d}_{MLE}$ in Figure 2.

Seeding performance. We compare our seeding algorithm Qkmeans with AFKMC² (Bachem et al., 2016a), RejectionSampling (Cohen-Addad et al., 2020a), PRONECoreset (Charikar et al., 2023) and FastKMeans (Draganov et al., 2024). All the algorithms were implemented in C++ without using any dimensionality reduction to ensure a fair comparison. Across a wide range of datasets and data modalities, the experiments show that Qkmeans consistently provides the fastest seeding procedure in practically relevant regimes, particularly as the number of clusters k and the ambient dimension D increase. RejectionSampling runtimes are almost independent of k , but it along with FastKMeans suffers a significant overhead due to constructing the multi-tree embeddings. AFKMC² is competitive only in low-dimensional tabular settings, but

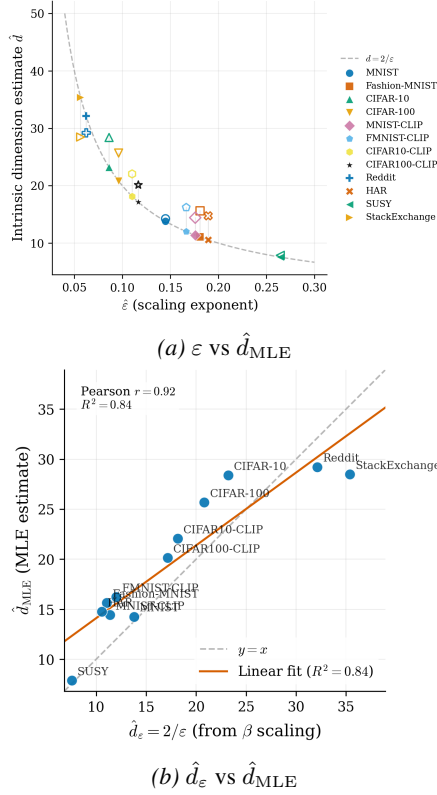


Figure 2. Dependence of quantization exponent on ID

loses its advantage on image, text, and embedding data. From the detailed benchmarks in Appendix D, Qkmeans achieves consistent speedups of 5–10 $\times$ over AFKMC² and PRONECoreset and up to two orders of magnitude over RejectionSampling, while maintaining comparable seeding cost. Additional experiments on the effect of noisy manifolds and studying the rejection rate vs m are presented in Appendix D.

4. Conclusion, Limitations, Future Directions

In this work, we modelled typical k -means clustering instances from a manifold hypothesis perspective. Using techniques from optimum quantization theory, we identified key geometric properties of the dataset that follow predictable scaling laws. We also showed how to exploit these regularities to design a fast rejection sampling-based seeding algorithm with new tradeoffs. Our theoretical results were backed by extensive empirical evaluations. We feel that due to its broad impact on the machine learning community, studying algorithms under the manifold hypothesis is an appealing and exciting direction for future research.

Impact Statement

This paper presents work whose goal is to advance the field of machine learning. There are many potential societal consequences of our work, none of which we feel must be

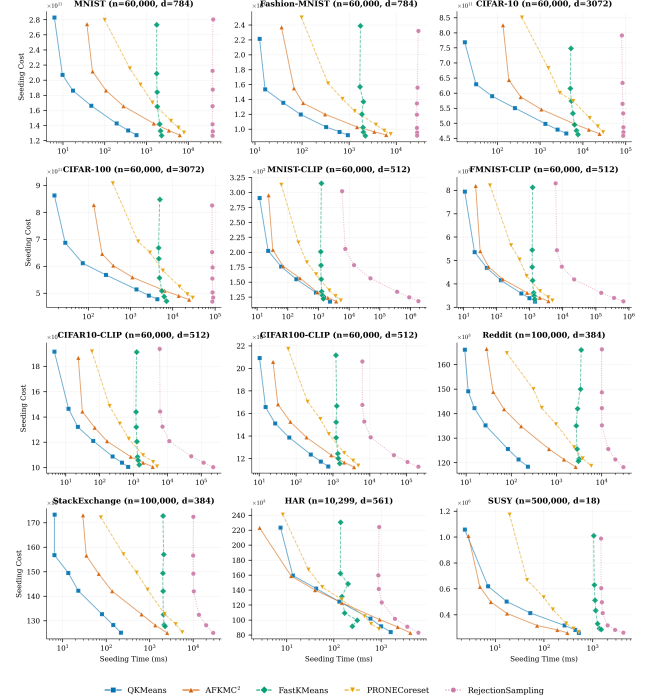


Figure 3. Seeding quality - runtime tradeoff plots

specifically highlighted here.

Acknowledgements

RJ would like to thank IIT Delhi - Abu Dhabi for hosting him during the later stages of this work. PS would like to acknowledge that this work was partially supported by the CSE Research Acceleration Fund of IIT Delhi.

References

- Agarwal, M., Jaiswal, R., and Pal, A. k -means++ under approximation stability. *Theoretical Computer Science*, 588:37–51, 2015. ISSN 0304-3975. doi: <https://doi.org/10.1016/j.tcs.2015.04.030>. URL <https://www.sciencedirect.com/science/article/pii/S0304397515003734>. 2
- Andoni, A. and Indyk, P. Near-optimal hashing algorithms for approximate nearest neighbor in high dimensions. *Communications of the ACM*, 51(1):117–122, 2008. 23
- Arthur, D. and Vassilvitskii, S. Worst-case and smoothed analysis of the icp algorithm, with an application to the k -means method. In *Symposium on Foundations of Computer Science*, 2006a. 2
- Arthur, D. and Vassilvitskii, S. How slow is the k -means method? In *SCG '06: Proceedings of the twenty-second annual symposium on computational geometry*. ACM Press, 2006b. 2
- Arthur, D. and Vassilvitskii, S. k -means++: the advantages of careful seeding. In *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, SODA '07, pp. 1027–1035, USA, 2007. Society for Industrial and Applied Mathematics. ISBN 9780898716245. 2, 25, 26
- Awasthi, P., Blum, A., and Sheffet, O. Stability yields a ptas for k -median and k -means clustering. In *Proceedings of the 2010 IEEE 51st Annual Symposium on Foundations of Computer Science*, FOCS '10, pp. 309–318, USA, 2010. IEEE Computer Society. ISBN 9780769542447. doi: 10.1109/FOCS.2010.36. URL <https://doi.org/10.1109/FOCS.2010.36>. 2
- Bachem, O., Lucic, M., Hassani, H., and Krause, A. Fast and provably good seedings for k -means. In Lee, D., Sugiyama, M., Luxburg, U., Guyon, I., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems*, volume 29. Curran Associates, Inc., 2016a. URL https://proceedings.neurips.cc/paper_files/paper/2016/file/d67d8ab4f4c10bf22aa353e27879133c-Paper.pdf. 4, 6, 8, 37
- Bachem, O., Lucic, M., Hassani, S. H., and Krause, A. Approximate k -means++ in sublinear time. In *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, AAAI'16, pp. 1459–1467. AAAI Press, 2016b. 4
- Bachem, O., Lucic, M., and Krause, A. Distributed and provably good seedings for k -means in constant rounds. In *ICML*, pp. 292–300, 2017a. 4
- Bachem, O., Lucic, M., and Krause, A. Practical core-set constructions for machine learning. *arXiv preprint arXiv:1703.06476*, 2017b. 5
- Bachem, O., Lucic, M., and Krause, A. Scalable k -means clustering via lightweight coresets. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, KDD '18, pp. 1119–1127, New York, NY, USA, 2018. Association for Computing Machinery. ISBN 9781450355520. doi: 10.1145/3219819.3219973. URL <https://doi.org/10.1145/3219819.3219973>. 4, 6
- Bahmani, B., Moseley, B., Vattani, A., Kumar, R., and Vassilvitskii, S. Scalable k -means++. *arXiv preprint arXiv:1203.6402*, 2012. 4
- Balcan, M. Data-driven algorithm design. In Roughgarden, T. (ed.), *Beyond Worst-Case Analysis of Algorithms*. Cambridge University Press, 2021. 2
- Balcan, M., Blum, A., and Gupta, A. Approximate clustering without the approximation. In *Proceedings of the 20th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pp. 1068–1077. SIAM, 2009. 2
- Barabasi, A.-L. and Albert, R. Emergence of scaling in random networks. *Science*, 286(5439):509–512, October 1999. ISSN 1095-9203. doi: 10.1126/science.286.5439.509. URL <http://dx.doi.org/10.1126/science.286.5439.509>. 3
- Bengio, Y., Courville, A., and Vincent, P. Representation learning: A review and new perspectives. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 35(8):1798–1828, 2013. doi: 10.1109/TPAMI.2013.50. 2, 3
- Bhattacharya, A., Eube, J., Röglin, H., and Schmidt, M. Noisy, Greedy and Not so Greedy k -Means++. In *28th Annual European Symposium on Algorithms (ESA 2020)*, volume 173 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pp. 18:1–18:21, 2020. 1
- Bilu, Y. and Linial, N. Are stable instances easy? In *Innovations in Computer Science (ICS)*, pp. 332–341, 2010. 2
- Binnie, J. A. D., Dłotko, P., Harvey, J., Malinowski, J., and Yim, K. M. A survey of dimension estimation methods, 2025. URL <https://arxiv.org/abs/2507.13887>. 35
- Blum, A. Approximation stability and proxy objectives. In Roughgarden, T. (ed.), *Beyond Worst-Case Analysis of Algorithms*. Cambridge University Press, 2021. 2

- Bordt, S., Upadhyay, U., Akata, Z., and von Luxburg, U. The manifold hypothesis for gradient-based explanations. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2023. URL https://openaccess.thecvf.com/content/CVPRW2023/html/XAI/Bordt_The_Manifold_Hypothesis_for_Gradient-Based_Explanations_CVPRW_2023_paper.pdf. Open Access CVPRW version; experiments on MNIST, CIFAR10, medical images. 2
- Bottesch, T., Bühler, T., and Kächele, M. Speeding up k -means by approximating euclidean distances via block vectors. In *Proceedings of the 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pp. 2578–2586, New York, New York, USA, 2016. PMLR. 5
- Brown, B. C., Caterini, A. L., Ross, B. L., Cresswell, J. C., and Loaiza-Ganem, G. Verifying the union of manifolds hypothesis for image data. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=Rvee9CAX4fi>. 2
- Camastra, F. and Staiano, A. Intrinsic dimension estimation: Advances and open problems. *Information Sciences*, 328:26–41, 2016. ISSN 0020-0255. doi: <https://doi.org/10.1016/j.ins.2015.08.029>. URL <https://www.sciencedirect.com/science/article/pii/S0020025515006179>. 35
- Charikar, M., Henzinger, M., Hu, L., Vötsch, M., and Waingarten, E. Simple, scalable and effective clustering via one-dimensional projections. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=JIYdbHDonF>. 5, 6, 8, 37
- Cohen-Addad, V. A fast approximation scheme for low-dimensional k -means. In *Proceedings of the Twenty-Ninth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA '18*, pp. 430–440. Society for Industrial and Applied Mathematics, 2018. 1
- Cohen-Addad, V. and Schwiegelshohn, C. On the local structure of stable clustering instances. In *2017 IEEE 58th Annual Symposium on Foundations of Computer Science (FOCS)*, pp. 49–60, 2017. doi: 10.1109/FOCS.2017.14. 2
- Cohen-Addad, V., Klein, P. N., and Mathieu, C. Local search yields approximation schemes for $\$k$ -means and $\$k$ -median in euclidean and minor-free metrics. *SIAM Journal on Computing*, 48(2):644–667, 2019. doi: 10.1137/17M112717X. 1
- Cohen-Addad, V., Lattanzi, S., Norouzi-Fard, A., Sohler, C., and Svensson, O. Fast and accurate k -means++ via rejection sampling. In Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., and Lin, H. (eds.), *Advances in Neural Information Processing Systems*, volume 33, pp. 16235–16245. Curran Associates, Inc., 2020a. URL https://proceedings.neurips.cc/paper_files/paper/2020/file/babcff88f8be8c4795bd6f0f8cccca61-Paper.pdf. 4, 5, 6, 7, 8, 23, 37
- Cohen-Addad, V., Lattanzi, S., Norouzi-Fard, A., Sohler, C., and Svensson, O. Fast and accurate k -means++ via rejection sampling. In *NeurIPS*, 2020b. 24
- Cornelissen, A., Doriguello, J. F., Luongo, A., and Tang, E. Do you know what q -means?, 2025. URL <https://arxiv.org/abs/2308.09701>. 6
- Curtin, R. R. A dual-tree algorithm for fast k -means clustering with large k . In *Proceedings of the 2017 SIAM International Conference on Data Mining*, pp. 300–308. SIAM, 2017. 5
- Dasgupta, S. How fast is k -means? In Schölkopf, B. and Warmuth, M. K. (eds.), *COLT*, volume 2777 of *Lecture Notes in Computer Science*, pp. 735. Springer, 2003. 2, 27
- Dasgupta, S. The hardness of k -means clustering. Technical report, UC San Diego: Department of Computer Science & Engineering, 2008. URL <https://escholarship.org/uc/item/2qm3k10c>. 1
- Dasgupta, S., 2013. URL <https://cseweb.ucsd.edu/~dasgupta/291-geom/kmeans.pdf>. CSE 291 : Geometric Algorithms, Lecture 3 - Algorithms for k -means clustering. 27
- Dasgupta, S. and Freund, Y. Random projection trees and low dimensional manifolds. In *Proceedings of the 40th Annual ACM Symposium on Theory of Computing*, pp. 537–546, 2008. 37
- Ding, Y., Zhao, Y., Shen, X., Musuvathi, M., and Mytkowicz, T. Yinyang k -means: A drop-in replacement of the classic k -means with consistent speedup. In *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pp. 579–587, Lille, France, 2015. PMLR. 5
- Draganov, A., Saulpic, D., and Schwiegelshohn, C. Settling time vs. accuracy tradeoffs for clustering big data. *Proc. ACM Manag. Data*, 2(3), May 2024.

- doi: 10.1145/3654976. URL <https://doi.org/10.1145/3654976>. 5, 6, 8, 37
- Elkan, C. Using the triangle inequality to accelerate k -means. In *Proceedings of the Twentieth International Conference on Machine Learning, ICML '03*, pp. 147–153. AAAI Press, 2003. 5
- Fefferman, C., Mitter, S., and Narayanan, H. Testing the manifold hypothesis. *Journal of the American Mathematical Society*, 29(4):983–1049, 2016. Rigorous theoretical treatment of the manifold hypothesis and testability. 2
- Feldman, D. Introduction to core-sets: An updated survey. *arXiv preprint arXiv:2011.09384*, 2020. 5
- Friggstad, Z., Rezapour, M., and Salavatipour, M. R. Local search yields a ptas for k -means in doubling metrics. *SIAM Journal on Computing*, 48(2):452–480, 2019. doi: 10.1137/17M1127181. 1
- Graf, S. and Luschgy, H. *Foundations of Quantization for Probability Distributions*. Lecture Notes in Mathematics. Springer Berlin Heidelberg, 2000. ISBN 9783540673941. URL <https://books.google.co.in/books?id=R1alK6sKL8QC>. 3, 15
- Gray, R. M. and Neuhoff, D. L. Quantization. *IEEE Trans. Inf. Theor.*, 44(6):2325–2383, September 2006. ISSN 0018-9448. doi: 10.1109/18.720541. URL <https://doi.org/10.1109/18.720541>. 3, 15
- Gruber, P. M. Optimum quantization and its applications. *Advances in Mathematics*, 186(2):456–497, 2004. doi: 10.1016/j.aim.2003.07.017. URL <https://doi.org/10.1016/j.aim.2003.07.017>. 3, 15
- Grunau, C., Özüdoğru, A. A., and Rozhoň, V. Noisy k -means++ revisited. In *ESA*, 2023. 7
- Hamerly, G. Making k -means even faster. In *Proceedings of the 2010 SIAM International Conference on Data Mining*, pp. 130–140. SIAM, 2010. 5
- Har-Peled, S. and Sadri, B. How fast is the k -means method? In *SODA '05: Proceedings of the sixteenth annual ACM-SIAM symposium on Discrete algorithms*, pp. 877–885, Philadelphia, PA, USA, 2005. Society for Industrial and Applied Mathematics. 2
- Horvat, C. and Pfister, J.-P. Estimating the intrinsic dimensionality using normalizing flows. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/34e3c2e99e927b19c9af9eab2f5e2e51-Abstract.html. Proposes flow-based estimators for intrinsic dimension via Jacobian spectrum analysis. 3
- Indyk, P. and Motwani, R. Approximate nearest neighbors: Towards removing the curse of dimensionality. In *Proceedings of the Thirtieth Annual ACM Symposium on Theory of Computing*, pp. 604–613, 1998. 37
- Jaiswal, R. and Garg, N. Analysis of k -means++ for separable data. In Gupta, A., Jansen, K., Rolim, J., and Servedio, R. (eds.), *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques*, pp. 591–602, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg. ISBN 978-3-642-32512-0. 2
- Jaiswal, R., Kumar, A., and Sen, S. A simple D2-sampling based PTAS for k -means and other clustering problems. *Algorithmica*, 70(1):22–46, 2014. doi: 10.1007/s00453-013-9833-9. 1
- Jaiswal, R., Kumar, M., and Yadav, P. Improved analysis of D2-sampling based PTAS for k -means and other clustering problems. *Information Processing Letters*, 115(2): 100–103, 2015. doi: 10.1016/j.ipl.2014.09.018. 1
- Jégou, H., Douze, M., and Schmid, C. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1):117–128, 2011a. doi: 10.1109/TPAMI.2010.57. 1
- Jégou, H., Douze, M., and Schmid, C. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1):117–128, 2011b. 37
- Johnson, J., Douze, M., and Jégou, H. Billion-scale similarity search with gpus. *arXiv preprint*, 2017. URL <https://arxiv.org/abs/1702.08734>. FAISS — practical ANN system using k -means / quantization techniques. 1
- Kanungo, T., Mount, D. M., Netanyahu, N. S., Piatko, C. D., Silverman, R., and Wu, A. Y. An efficient k -means clustering algorithm: Analysis and implementation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 24(7):881–892, 2002. 5
- Kumar, A. and Kannan, R. Clustering with spectral norm and the k -means algorithm. In *FOCS*, pp. 299–308. IEEE, 2010. 2
- Kumar, A., Sabharwal, Y., and Sen, S. Linear-time approximation schemes for clustering problems in any dimensions. *J. ACM*, 57(2):5:1–5:32, 2010. doi: 10.1145/1667053.1667054. 1
- Lee, J. M. *Introduction to Riemannian Manifolds*. Graduate Texts in Mathematics. Springer Cham, 2 edition, 2019. ISBN 978-3-319-91754-2. doi: 10.1007/978-3-319-91755-9. 15

- Levina, E. and Bickel, P. J. Maximum likelihood estimation of intrinsic dimension. *Advances in Neural Information Processing Systems*, 17, 2005. 4, 8, 35, 36
- Linde, Y., Buzo, A., and Gray, R. M. An algorithm for vector quantizer design. *IEEE Transactions on Communications*, 28(1):84–95, 1980. doi: 10.1109/TCOM.1980.1094574. 1
- Lloyd, S. P. Least squares quantization in pcm. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982. doi: 10.1109/TIT.1982.1056489. Originally presented as a Bell Labs Technical Report in 1957. 1
- Loaiza-Ganem, G., Ross, B. L., Hosseinzadeh, R., Caterini, A. L., and Cresswell, J. C. Deep generative models through the lens of the manifold hypothesis: A survey and new connections. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856. URL <https://openreview.net/forum?id=a90WpmSi0I>. Survey Certification, Expert Certification. 3
- Mahajan, M., Nimbhorkar, P., and Varadarajan, K. The planar k -means problem is np-hard. In *Proceedings of the 3rd International Workshop on Algorithms and Computation*, WALCOM '09, pp. 274–285, Berlin, Heidelberg, 2009. Springer-Verlag. ISBN 9783642002014. doi: 10.1007/978-3-642-00202-1_24. URL https://doi.org/10.1007/978-3-642-00202-1_24. 1
- Makarychev, K., Makarychev, Y., and Razenshteyn, I. Performance of johnson-lindenstrauss transform for k -means and k -medians clustering. In *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing*, STOC 2019, pp. 1027–1038, New York, NY, USA, 2019. Association for Computing Machinery. ISBN 9781450367059. doi: 10.1145/3313276.3316350. URL <https://doi.org/10.1145/3313276.3316350>. 5, 22, 33
- Malkov, Y. A. and Yashunin, D. A. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020. 37
- Newling, J. and Fleuret, F. Fast k -means with accurate bounds. In *Proceedings of the 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pp. 936–944, New York, New York, USA, 2016. PMLR. 5
- Ostrovsky, R., Rabani, Y., Schulman, L. J., and Swamy, C. The effectiveness of lloyd-type methods for the k -means problem. *Journal of the ACM*, 59(6):1–22, 2013. 2
- Pelleg, D. and Moore, A. W. Accelerating exact k -means algorithms with geometric reasoning. In *Proceedings of the Fifth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '99, pp. 277–281, New York, NY, USA, 1999. ACM. 5
- Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., et al. Learning transferable visual models from natural language supervision. In *ICML*, 2021. 8
- Roughgarden, T. Beyond worst-case analysis. *Commun. ACM*, 62(3):88–96, February 2019. ISSN 0001-0782. doi: 10.1145/3232535. URL <https://doi.org/10.1145/3232535>. 2
- Shah, P. C. and Jaiswal, R. Quantum (inspired) Sd^2S -sampling with applications. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=tDIL7UXmSS>. 6
- Shamos, M. I. and Hoey, D. Closest-point problems. In *16th Annual Symposium on Foundations of Computer Science, Berkeley, California, USA, October 13-15, 1975*, pp. 151–162. IEEE Computer Society, 1975. doi: 10.1109/SFCS.1975.8. URL <https://doi.org/10.1109/SFCS.1975.8>. 4
- Tang, E. A quantum-inspired classical algorithm for recommendation systems. In *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing*, STOC 2019, pp. 217–228, New York, NY, USA, 2019. Association for Computing Machinery. ISBN 9781450367059. doi: 10.1145/3313276.3316310. URL <https://doi.org/10.1145/3313276.3316310>. 23
- Tempczyk, P., Michaluk, R., Garncarek, L., Spurek, P., Tabor, J., and Golinski, A. LIDL: Local intrinsic dimension estimation using approximate likelihood. In Chaudhuri, K., Jegelka, S., Song, L., Szepesvari, C., Niu, G., and Sabato, S. (eds.), *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pp. 21205–21231. PMLR, 17–23 Jul 2022. URL <https://proceedings.mlr.press/v162/tempczyk22a.html>. 3
- Wang, J., Wang, J., Ke, Q., Zeng, G., and Li, S. Fast approximate k -means via cluster closures. In *2012 IEEE Conference on Computer Vision and Pattern Recognition*, pp. 3037–3044. IEEE, 2012. 5
- Wang, W., Wei, F., Dong, L., Bao, H., Yang, N., and Zhou, M. Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained

transformers. In *Advances in Neural Information Processing Systems*, volume 33, pp. 5776–5788, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>. 8

Wu, X., Kumar, V., Quinlan, J. R., Ghosh, J., Yang, Q., Motoda, H., McLachlan, G. J., Ng, A., Liu, B., Yu, P. S., Zhou, Z.-H., Steinbach, M., Hand, D. J., and Steinberg, D. Top 10 algorithms in data mining. *Knowledge and Information Systems*, 14(1):1–37, 2008. ISSN 0219-3116. doi: 10.1007/s10115-007-0114-2. URL <https://link.springer.com/article/10.1007/s10115-007-0114-2>. 1

Zador, P. L. Asymptotic quantization error of continuous signals and the quantization dimension. *IEEE Transactions on Information Theory*, 28(2):139–149, 1982. doi: 10.1109/TIT.1982.1056489. 3, 15

Zipf, G. K. *Human Behavior and the Principle of Least Effort*. Addison-Wesley, Cambridge, MA, 1949. 3

Appendix

The appendices expand upon the content of the main paper. In Appendix A, we provide a theoretical proof of the scaling laws presented in Theorem 5. In Appendix C we provide the technical proofs for our seeding algorithm along with the analysis of its runtime and approximation guarantees. Finally, in Appendix D we provide the details of our empirical study along with additional experiments and data.

A. Theoretical Analysis of Scaling Laws

In this section we provide theoretical proofs for the scaling laws presented in Theorem 5. We mention that the techniques used here are well developed in optimum quantization theory (Zador, 1982; Gray & Neuhoff, 2006; Graf & Luschgy, 2000; Gruber, 2004) and we provide an elementary proof for the sake of being self-contained. Since we work under the manifold assumption, we require definitions and techniques from differential geometry for our proofs. We refer the reader to the book by (Lee, 2019) for obtaining familiarity with such notions. Recall that we are given a dataset $\mathcal{X} = \{x_1, \dots, x_n\}$ where the x_i 's are sampled i.i.d. from a probability density function $f : \mathcal{M} \rightarrow \mathbb{R}_+$ supported on a compact, connected and smooth subset of a manifold $\mathcal{M} \subset \mathbb{R}^D$. The proof can be broken down into the following two steps :

- *Step 1* Showing that the optimum distortion $\Delta_k(f) = \inf_{|C|=k} \int_{\mathcal{M}} \min_{c \in C} \|x - c\|^2 f(x) \, \text{dvol}_{\mathcal{M}}(x)$ follows the scaling law $\Delta_k(f) \in \Theta_f(k^{-2/d})$.
- *Step 2* Showing how to translate this in the finite sample case by studying how $\frac{1}{n} \text{opt}_k(\mathcal{X}) \rightarrow \Delta_k(f)$ as $n \rightarrow \infty$.

Preliminaries. A function $\varphi : A \rightarrow B$ between two metric spaces A and B is called a *homeomorphism* if φ is bijective and both φ and φ^{-1} are continuous. A set $\mathcal{M} \subset \mathbb{R}^D$ is called a smooth d -dimensional manifold embedded in $\mathbb{R}^D$ if for each point $p \in \mathcal{M}$ there exists an open neighborhood $W \subset \mathbb{R}^D$ of p and a smooth diffeomorphism $\varphi : W \rightarrow \mathbb{R}^d$ such that $\varphi(\mathcal{M} \cap W) = \mathbb{R}^d \times \{0\}^{D-d}$. Equivalently, $\mathcal{M}$ admits a collection of local coordinate maps as follows. A *chart* on $\mathcal{M}$ is a pair (U, φ) , where $U \subset \mathcal{M}$ is open and $\varphi : U \rightarrow V \subset \mathbb{R}^d$ is a homeomorphism onto an open subset of $\mathbb{R}^d$. A collection of charts $\mathcal{A} = \{(U_\alpha, \varphi_\alpha)\}_{\alpha \in I}$ is called an *atlas* if $\mathcal{M} = \bigcup_{\alpha \in I} U_\alpha$. The atlas is said to be *smooth* if for every pair of overlapping charts $(U_\alpha, \varphi_\alpha)$ and (U_β, φ_β) , the transition maps $\varphi_\beta \circ \varphi_\alpha^{-1} : \varphi_\alpha(U_\alpha \cap U_\beta) \rightarrow \varphi_\beta(U_\alpha \cap U_\beta)$ are smooth diffeomorphisms. Throughout, we assume that $\mathcal{M}$ is equipped with the smooth atlas induced by its embedding in $\mathbb{R}^D$. This atlas endows $\mathcal{M}$ with the structure of a smooth manifold and allows us to perform local analysis by identifying sufficiently small neighborhoods of $\mathcal{M}$ with open subsets of $\mathbb{R}^d$.

Lemma 19 (Zador bounds for manifolds). *Let $\mathcal{M} \subset \mathbb{R}^D$ be a compact, d -dimensional C^2 submanifold and let $J \subset \mathcal{M}$ be compact, measurable, and satisfy $\text{vol}_{\mathcal{M}}(J) > 0$. Let $f : J \rightarrow \mathbb{R}_+$ be a probability density function on $\mathcal{M}$ such that $f_{\min} < f(x) < f_{\max}$ and $\int_{\mathcal{M}} f(x) \, \text{dvol}_{\mathcal{M}}(x) = 1$. Define*

$$\Delta_k(f) := \inf_{\substack{S \subset \mathbb{R}^D \\ |S|=k}} \int_{\mathcal{M}} \min_{p \in S} \|x - p\|^2 f(x) \, \text{dvol}_{\mathcal{M}}(x).$$

Then there exist constants $0 < c < C < \infty$ depending only on $\mathcal{M}, J$ and f such that

$$c k^{-2/d} \leq \Delta_k(f) \leq C k^{-2/d} \quad \text{for all } k \geq 1.$$

Proof. Upper bound. Since J is compact and $\mathcal{M}$ is a smooth embedded submanifold, there exists a finite atlas $\{(U_\ell, \varphi_\ell)\}_{\ell=1}^m$ such that

$$J = \bigcup_{\ell=1}^m J_\ell, \quad J_\ell \subset U_\ell,$$

where each $\varphi_\ell : U_\ell \rightarrow V_\ell \subset \mathbb{R}^d$ is a diffeomorphism. By compactness and smoothness, shrinking charts if necessary, there exists a constant $C_0 \geq 1$ such that for all ℓ and all $x, y \in U_\ell$,

$$C_0^{-1} \|\varphi_\ell(x) - \varphi_\ell(y)\| \leq \|x - y\| \leq C_0 \|\varphi_\ell(x) - \varphi_\ell(y)\|, \quad (1)$$

and the induced volume form satisfies

$$C_0^{-1} du \leq \text{dvol}_{\mathcal{M}}(x) \leq C_0 du \quad \text{under } u = \varphi_\ell(x). \quad (2)$$

Fix $k \geq m$ and set $k_\ell := \lfloor k/m \rfloor$. The set $\varphi_\ell(J_\ell) \subset \mathbb{R}^d$ is compact, hence contained in a cube of side length $L_\ell < \infty$. Partition this cube into $\lceil k_\ell^{1/d} \rceil^d$ congruent subcubes, each of side length at most $2L_\ell k_\ell^{-1/d}$. Selecting one point from each subcube intersecting $\varphi_\ell(J_\ell)$ yields a set $\{u_{\ell,i}\}_{i=1}^{k_\ell} \subset \varphi_\ell(J_\ell)$ such that

$$\varphi_\ell(J_\ell) \subset \bigcup_{i=1}^{k_\ell} B_{\mathbb{R}^d}(u_{\ell,i}, Ck^{-1/d}),$$

for a constant C depending only on J and d .

Define $p_{\ell,i} := \varphi_\ell^{-1}(u_{\ell,i}) \in J_\ell$ and let

$$S := \bigcup_{\ell=1}^m \{p_{\ell,1}, \dots, p_{\ell,k_\ell}\},$$

so that $|S| \leq k$. By (1), for every $x \in J$,

$$\min_{p \in S} \|x - p\| \leq C' k^{-1/d},$$

and hence

$$\min_{p \in S} \|x - p\|^2 \leq C'' k^{-2/d}.$$

Integrating over J and using the finiteness of $\text{vol}_{\mathcal{M}}(J)$ gives

$$\int_J \min_{p \in S} \|x - p\|^2 f(x) \, d\text{vol}_{\mathcal{M}}(x) \leq Ck^{-2/d},$$

which implies $\Delta_k(f) \leq Ck^{-2/d}$.

Lower bound. Since $\text{vol}_{\mathcal{M}}(J) > 0$ and $\mathcal{M}$ is smooth, there exists a point $x_0 \in J$ and constants $\varrho > 0$ and $c_0 > 0$ such that the geodesic ball $B_{\mathcal{M}}(x_0, r) \subset J$ for all $0 < r \leq \varrho$, and

$$\text{vol}_{\mathcal{M}}(B_{\mathcal{M}}(x_0, r)) \geq c_0 r^d. \quad (3)$$

Let $S \subset \mathbb{R}^D$ with $|S| = k$ be arbitrary and let $r > 0$. For each $p \in S$, the intersection $B(p, r) \cap \mathcal{M}$ is contained in a geodesic ball of radius Cr , hence by (3),

$$\text{vol}_{\mathcal{M}}(B(p, r) \cap \mathcal{M}) \leq Cr^d.$$

Therefore,

$$\text{vol}_{\mathcal{M}}\left(\bigcup_{p \in S} B(p, r) \cap \mathcal{M}\right) \leq Ckr^d.$$

Choosing $r = ck^{-1/d}$ with $c > 0$ sufficiently small ensures that

$$\text{vol}_{\mathcal{M}}\left(\bigcup_{p \in S} B(p, r) \cap \mathcal{M}\right) < \text{vol}_{\mathcal{M}}(B_{\mathcal{M}}(x_0, \varrho)).$$

Hence there exists a measurable subset

$$A \subset B_{\mathcal{M}}(x_0, \varrho)$$

of positive volume such that $\|x - p\| \geq r$ for all $x \in A$ and all $p \in S$. For such x ,

$$\min_{p \in S} \|x - p\|^2 \geq r^2 = c^2 k^{-2/d}.$$

Integrating over A yields

$$\int_J \min_{p \in S} \|x - p\|^2 f(x) \, d\text{vol}_{\mathcal{M}}(x) \geq c' k^{-2/d},$$

where $c' > 0$ depends only on $\mathcal{M}$ and J . Since S was arbitrary, this implies $\Delta_k(f) \geq ck^{-2/d}$.

Combining the upper and lower bounds completes the proof. $\square$

Next, we show finite sample bounds on the k -means cost to quantify how $\text{opt}_k(\mathcal{X}) \rightarrow \Delta_k(f)$ as the number of samples $n \rightarrow \infty$. By re-scaling, we can assume without loss of generality that $\|x\| \leq 1$ for each $x \in J$. For this, we would require two standard results :

Lemma 20 (Hoeffding's Inequality). *Let $X_1, \dots, X_m$ be independent random variables such that $a_j \leq X_j \leq b_j$ almost surely for each $j \in [m]$. Let $S = \sum_{j=1}^m X_j$ then :*

$$\Pr[|S_m - \mathbf{E}[S_m]| \geq t] \leq 2 \exp\left(-\frac{2t^2}{\sum_{j=1}^m (b_j - a_j)^2}\right)$$

Lemma 21 (Covering Lemma). *Let $A \subset \mathbb{R}^m$ be a bounded set. Then for any $\delta > 0$, there exists a finite set $\mathcal{N}_\delta(A) \subset A$, called a δ -net for A , such that:*

1. *For every $x \in A$, there exists $x' \in \mathcal{N}_\delta(A)$ with $\|x - x'\| \leq \delta$.*
2. *$|\mathcal{N}_\delta(A)| \leq \left(\frac{\text{diam}(A)}{\delta} + 1\right)^m$.*

Using these, we can show the following :

Lemma 22 (Finite sample bound on k -means cost). *For any $\delta \in (0, 1)$, the following holds with probability atleast $1 - \delta$:*

$$\left| \frac{1}{n} \text{opt}_k(\mathcal{X}) - \Delta_k(f) \right| \in O_f \left(\sqrt{\frac{kD \log n + \log(1/\delta)}{n}} \right)$$

Proof. Let us define the set of possible centers $\mathcal{C}_k := \{(c_1, \dots, c_k) \subset \mathbb{R}^D : \|c_j\| \leq 1, 1 \leq j \leq k\}$. For a fixed set of centers $C = (c_1, \dots, c_k) \in \mathcal{C}_k$, let us define the following quantities:

$$\begin{aligned} \ell(x|C) &:= \min_{1 \leq j \leq k} \|x - c_j\|^2 \\ L_n(C) &:= \frac{1}{n} \sum_{i \leq i \leq n} \ell(x_i|C) \\ L(C) &:= \mathbf{E}_{x \sim f} \ell(x|C) \end{aligned}$$

It can be easily seen that the function $\ell(x|C)$ is bounded since:

$$\ell(x|C) = \min_{1 \leq j \leq k} \|x - c_j\|^2 \leq \|x - c_1\|^2 \leq 2(\|x\|^2 + \|c_1\|^2) \leq 4$$

Notice that by definition, $L_n(C)$ is the average of n i.i.d. random variables $\ell(x_i|C)$ each of which is bounded in $[0, 4]$. Hence, applying Lemma 20 implies that for any $t > 0$,

$$\Pr(|L_n(C) - L(C)| \geq t) \leq 2 \exp(-nt^2/8)$$

We next control the dependence of the loss on the centers. Let $C = (c_1, \dots, c_k)$ and $C' = (c'_1, \dots, c'_k)$ be two elements of $\mathcal{C}_k$ such that

$$\max_{j \in [k]} \|c_j - c'_j\| \leq \delta.$$

For any $x \in \mathbb{R}^D$, we have

$$\begin{aligned} |\ell(x|C) - \ell(x|C')| &= \left| \min_{1 \leq j \leq k} \|x - c_j\|^2 - \min_{1 \leq j \leq k} \|x - c'_j\|^2 \right| \\ &\leq \max_{1 \leq j \leq k} \left| \|x - c_j\|^2 - \|x - c'_j\|^2 \right|. \end{aligned}$$

For a fixed j , we may expand

$$\begin{aligned} \left| \|x - c_j\|^2 - \|x - c'_j\|^2 \right| &= \left| \langle c'_j - c_j, 2x - c_j - c'_j \rangle \right| \\ &\leq \|c_j - c'_j\| \cdot \|2x - c_j - c'_j\|. \end{aligned}$$

Since $\|x\| \leq 1$ and $\|c_j\|, \|c'_j\| \leq 1$, it follows that

$$\|2x - c_j - c'_j\| \leq 2\|x\| + \|c_j\| + \|c'_j\| \leq 4.$$

Consequently,

$$|\ell(x | C) - \ell(x | C')| \leq 4 \max_{j \in [k]} \|c_j - c'_j\| \leq 4\delta.$$

Averaging over the samples and taking expectation yields

$$|L_n(C) - L_n(C')| \leq 4\delta, \quad |L(C) - L(C')| \leq 4\delta.$$

Let $\varepsilon > 0$ be a parameter to be chosen later. By Lemma 21, there exists an ε -net $\mathcal{N}_\varepsilon \subset \mathcal{C}_k$ such that

$$|\mathcal{N}_\varepsilon| \leq \left(\frac{2}{\varepsilon} + 1\right)^{kD} \leq \left(\frac{3}{\varepsilon}\right)^{kD}.$$

For any $C \in \mathcal{C}_k$, let $\Pi(C) \in \mathcal{N}_\varepsilon$ denote a nearest net point. Then

$$|L_n(C) - L(C)| \leq |L_n(\Pi(C)) - L(\Pi(C))| + 8\varepsilon.$$

Applying Hoeffding's inequality and a union bound over $\mathcal{N}_\varepsilon$, we obtain for any $t > 0$,

$$\Pr\left(\sup_{C' \in \mathcal{N}_\varepsilon} |L_n(C') - L(C')| \geq t\right) \leq 2|\mathcal{N}_\varepsilon| \exp(-nt^2/8).$$

Thus, with probability at least $1 - \delta$,

$$\sup_{C' \in \mathcal{N}_\varepsilon} |L_n(C') - L(C')| \lesssim \sqrt{\frac{kD \log(1/\varepsilon) + \log(1/\delta)}{n}}.$$

Combining the above bounds, we conclude that with probability at least $1 - \delta$,

$$\sup_{C \in \mathcal{C}_k} |L_n(C) - L(C)| \lesssim \sqrt{\frac{kD \log(1/\varepsilon) + \log(1/\delta)}{n}} + \varepsilon.$$

Choosing $\varepsilon = 1/n$ gives $\log(1/\varepsilon) = \log n$, and hence

$$\sup_{C \in \mathcal{C}_k} |L_n(C) - L(C)| \in O\left(\sqrt{\frac{kD \log n + \log(1/\delta)}{n}}\right).$$

Finally, observing that

$$\frac{1}{n} \text{opt}_k(\mathcal{X}) = \inf_{C \in \mathcal{C}_k} L_n(C), \quad \Delta_k(f) = \inf_{C \in \mathcal{C}_k} L(C),$$

we obtain

$$\left| \frac{1}{n} \text{opt}_k(\mathcal{X}) - \Delta_k(f) \right| \leq \sup_{C \in \mathcal{C}_k} |L_n(C) - L(C)|,$$

which completes the proof. $\square$

We are now ready to provide a proof for Theorem 5.

Theorem 23 (Scaling laws). *Suppose $\mathcal{X} = \{x_1, \dots, x_n\} \subset \mathbb{R}^D$ consists of i.i.d. samples drawn from a distribution f supported on a compact, smooth d -dimensional manifold $\mathcal{M} \subset \mathbb{R}^D$, with density bounded above and below on its support. Let $\varepsilon := 2/d$. Then, with probability at least $1 - \frac{1}{\text{poly}(n)}$, the following hold:*

$$\beta_k(\mathcal{X}) = \left(1 + O_f\left(\sqrt{\frac{kD \log n}{n}}\right)\right) k^\varepsilon, \quad \eta(\mathcal{X}) \in O_f(1) n^{3\varepsilon/2}.$$

Proof. We prove the two statements separately.

Analysis of $\beta_k(\mathcal{X})$. By definition,

$$\beta_k(\mathcal{X}) = \frac{\text{opt}_1(\mathcal{X})}{\text{opt}_k(\mathcal{X})}, \quad \text{opt}_k(\mathcal{X}) = \min_{C:|C|=k} \sum_{i=1}^n \min_{c \in C} \|x_i - c\|^2.$$

Define the normalized empirical distortion

$$\frac{1}{n} \text{opt}_k(\mathcal{X}).$$

By Lemma 22, for any fixed k , with probability at least $1 - \frac{1}{\text{poly}(n)}$,

$$\left| \frac{1}{n} \text{opt}_k(\mathcal{X}) - \Delta_k(f) \right| \leq C_f \sqrt{\frac{kD \log n}{n}}.$$

In particular, the same bound holds for $k = 1$ with the factor k removed.

On the other hand, by Zador's theorem on manifolds (Lemma 19 and its matching lower bound),

$$\Delta_1(f) = \Theta_f(1), \quad \Delta_k(f) = \Theta_f(k^{-2/d}).$$

Combining these estimates yields, with high probability,

$$\frac{1}{n} \text{opt}_1(\mathcal{X}) = \Theta_f(1) + O_f\left(\sqrt{\frac{D \log n}{n}}\right),$$

and

$$\frac{1}{n} \text{opt}_k(\mathcal{X}) = \Theta_f(k^{-2/d}) \left(1 + O_f\left(\sqrt{\frac{kD \log n}{n}}\right) \right),$$

where the error term is of lower order provided $n \gg k^{1+2/d} \log n$.

Taking the ratio, we obtain

$$\beta_k(\mathcal{X}) = \frac{\text{opt}_1(\mathcal{X})}{\text{opt}_k(\mathcal{X})} = \frac{\frac{1}{n} \text{opt}_1(\mathcal{X})}{\frac{1}{n} \text{opt}_k(\mathcal{X})} = \left(1 + O_f\left(\sqrt{\frac{kD \log n}{n}}\right) \right) k^{2/d},$$

which proves the first claim.

Analysis of $\eta(\mathcal{X})$. By definition,

$$\eta(\mathcal{X}) = \frac{\max_{i \neq j} \|x_i - x_j\|}{\min_{i \neq j} \|x_i - x_j\|}.$$

Since f is supported on a compact set, we have

$$\max_{i \neq j} \|x_i - x_j\| \leq \text{diam}(\text{supp } f) = O_f(1) \quad \text{almost surely.}$$

It remains to lower bound the minimum interpoint distance.

By the volume growth property of a smooth d -dimensional manifold and the lower bound on the density of f , there exists a constant $c_f > 0$ such that for all sufficiently small r and all $x \in \mathcal{M}$,

$$\Pr(\|X - x\| \leq r) \leq c_f r^d.$$

Applying a union bound over all unordered pairs (i, j) with $i \neq j$, we obtain

$$\Pr\left(\min_{i \neq j} \|x_i - x_j\| \leq r\right) \leq n^2 c_f r^d.$$

Choosing $r = n^{-3/d}$ yields

$$\Pr\left(\min_{i \neq j} \|x_i - x_j\| \leq r\right) \leq c_f n^2 \cdot \frac{1}{n^3} = \frac{1}{\text{poly}(n)},$$

for all sufficiently large n . Therefore, with probability at least $1 - \frac{1}{\text{poly}(n)}$,

$$\min_{i \neq j} \|x_i - x_j\| \geq c'_f n^{-3/d}$$

for some constant $c'_f > 0$. Combining this with the diameter bound yields

$$\eta(\mathcal{X}) = \frac{O_f(1)}{\Omega_f(n^{-3/d})} = O_f(n^{3/d}) = O_f(n^{3\varepsilon/2}).$$

□

B. Notation

#	Symbol	Meaning
1	n	number of points in the dataset
2	D	ambient dimension of the dataset
3	k	number of clusters into which the dataset is to be partitioned
4	$\mathcal{X} = \{x_1, \dots, x_n\}$	dataset in consideration
5	$\mu = \mu(\mathcal{X})$	mean of the dataset $\mathcal{X}$
6	$\ \mathcal{X}\ _F$	frobenius norm of the dataset defined as $\ \mathcal{X}\ _F^2 = \sum_{i=1}^n \ x_i\ ^2$
7	$C = \{c_1, \dots, c_k\}$	a set of k cluster centers
8	$\text{cost}(\mathcal{X}, C)$	k -means cost of dataset $\mathcal{X}$ with respect to C
9	$\pi(\cdot C)$	D^2 distribution with respect to centers C given by $\pi(x C) = \frac{\text{cost}(x, C)}{\text{cost}(\mathcal{X}, C)}$
10	$\kappa(\cdot C)$	proposal distribution with respect to center c_1 given by $\kappa(x C) = \frac{\ x\ ^2 + \ c_1\ ^2}{\ \mathcal{X}\ _F^2 + n\ c_1\ ^2}$
11	$\text{opt}_k(\mathcal{X})$	optimal k -means cost for dataset $\mathcal{X}$
12	$\{s_1, \dots, s_k\}$	an optimal set of cluster centers
13	$\mathcal{X} = \bigcup_{j=1}^k \mathcal{X}_j$	optimal clustering partition
14	$\mathcal{L} = \mathcal{L}_\varrho(C)$	LSH data structure initialized by parameter ϱ into which the centers C have been insert
15	$\text{Insert}(c, \mathcal{L})$	operation to insert a center c in $\mathcal{L}$
16	$\text{Query}(x, \mathcal{L})$	operation to query an approximate nearest neighbor of x from $\mathcal{L}$
17	$\text{cost}^\mathcal{L}(\mathcal{X}, C)$	defined as $\sum_{x \in \mathcal{X}} \text{cost}(x, \text{Query}(x, \mathcal{L}))$
18	$\pi^\mathcal{L}(\cdot C)$	$\pi^\mathcal{L}(x C) = \frac{\text{cost}^\mathcal{L}(x, C)}{\text{cost}^\mathcal{L}(\mathcal{X}, C)}$
19	t	during the t th iteration t centers have been chosen
20	$C_t = \{c_1, \dots, c_t\}$	centers chosen uptill t iterations
21	H_t, U_t, W_t	number of covered clusters, uncovered clusters and wasted iterations uptill iteration t
22	$\mathcal{X} = \mathcal{H}_t \cup \mathcal{U}_t$	partitioning of $\mathcal{X}$ into covered and uncovered clusters
23	$\text{cost}_t(\mathcal{P})$	defined as $\text{cost}(\mathcal{P}, C_t)$
24	d	dimension of the underlying manifold $\mathcal{M}$
25	ε	quantization exponent defined as $\varepsilon := 2/d$
26	ϱ	LSH parameter
27	γ	overall exponent defined as $\gamma := \varepsilon + \varrho$
28	f	probability density function supported on the underlying manifold $\mathcal{M}$

Table 3. Index of notation used throughout the appendix

C. Theoretical Analysis of Qkmeans

The goal of this section is to describe our proof techniques for the theoretical analysis of the Qkmeans seeding algorithm, which we reproduce here for convenience.

Algorithm 5 QKMEANS($\mathcal{X}, k, m$)

1. Preprocessing:

- (a) (Optional) For $\varepsilon_{\text{JL}} \in (0, 1/4)$, apply the JL-like dimensionality reduction from (Makarychev et al., 2019) to $\mathcal{X}$, reducing to dimension

$$D \in O\left(\frac{\log(k/\varepsilon_{\text{JL}})}{\varepsilon_{\text{JL}}^2}\right)$$

with distortion $(1 + \varepsilon_{\text{JL}})$.

- (b) Compute the mean $\mu \leftarrow \frac{1}{n} \sum_{x \in \mathcal{X}} x$.
 (c) Center each point: $x \leftarrow x - \mu$ for all $x \in \mathcal{X}$.
 (d) Compute $\|\mathcal{X}\|_{\text{F}}^2 \leftarrow \sum_{x \in \mathcal{X}} \|x\|^2$.
 (e) Compute $\kappa(x) \leftarrow \|x\|^2 / \|\mathcal{X}\|_{\text{F}}^2$ for all $x \in \mathcal{X}$.

2. Initialization:

- (a) Initialize the data structure $\text{INIT}(\mathcal{L}, \varrho)$.
 (b) Sample $c_1 \sim \text{UNIFORM}(\mathcal{X})$.
 (c) Set $C_1 \leftarrow \{c_1\}$ and insert c_1 into $\mathcal{L}$.

3. Main loop:

- (a) **for** $t = 2$ **to** k **do**
 i. Initialize $\text{iter} \leftarrow 0$ and sample $s \sim \text{UNIFORM}(\mathcal{X})$.
 ii. **while** $\text{iter} < m \ln k$ **do**
 A. $\text{iter} \leftarrow \text{iter} + 1$.
 B. Sample $x \sim \kappa(\cdot \mid C_{t-1})$, where

$$\kappa(x \mid C_{t-1}) := \frac{\|x\|^2 + \|c_1\|^2}{\|\mathcal{X}\|_{\text{F}}^2 + n\|c_1\|^2}.$$

C. Sample $R \sim \text{UNIFORM}(0, 1)$.

D. Compute

$$r \leftarrow \frac{\text{cost}(x, \text{Query}(\mathcal{L}, x))}{2\varrho^{-1}(\|x\|^2 + \|c_1\|^2)}.$$

E. **if** $R \leq r$ **then**

 Set $s \leftarrow x$. **break**.

- iii. Set $c_t \leftarrow s$, insert c_t into $\mathcal{L}$, and update

$$C_t \leftarrow C_{t-1} \cup \{c_t\}.$$

4. Return C_k .

C.1. Preprocessing

The Preprocessing step ($\text{PREPROCESS}(\mathcal{X})$, Algorithm 1) clearly takes a time of $O(nD)$ as it involves a JL-like linear transformation on the dataset, followed by a couple of for loops iterating over the transformed D -dimensional dataset of n points.

Consider the vector $v_{\mathcal{X}} \in \mathbb{R}^n$ given by $v_{\mathcal{X}}(x) = \|x\|^2$. Define $\kappa(x) = \frac{\|x\|^2}{\|\mathcal{X}\|_{\text{F}}^2}$ as a distribution over $\mathcal{X}$. We will use a (complete) binary tree data structure to sample from $\kappa(\cdot)$. The leaves of the binary tree correspond to the entries of $v_{\mathcal{X}}$ and

store weight $v_{\mathcal{X}}(x)^2$ along with the sign of $v_{\mathcal{X}}(x)$. The internal nodes also store a weight that is equal to the sum of weights of its children. To sample from $\kappa(\cdot)$, we traverse the tree, choosing either to go left or right at each node with probability proportional to the weight of its two children until reaching the leaves. The binary tree similarly supports querying and updating the entries of $v_{\mathcal{X}}$.

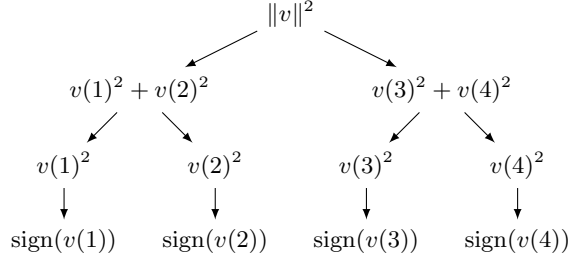


Figure 4. Data structure for sampling from a vector $v \in \mathbb{R}^4$

We state this formally following (Tang, 2019), in which such data structures, called *sample and query access data structures* were introduced.

Lemma 24. (Lemma 3.1 in (Tang, 2019)) *There exists a data structure storing a vector $v \in \mathbb{R}^n$ with ν nonzero entries in $O(\nu \log n)$ space which supports the following operations:*

- *Reading and updating an entry of v in $O(\log n)$ time*
- *Finding $\|v\|^2$ in $O(1)$ time*
- *Generating an independent sample $i \in \{1, \dots, n\}$ with probability $\frac{v(i)^2}{\sum_{j=1}^n v(j)^2}$ in $O(\log n)$ time.*

Note that if n is not a perfect power of 2 then we can find a $n' \in \mathbb{N}$ which is a perfect power of 2 such that $n' < n < 2n'$. We can then set the remaining $2n' - n$ data points to have zero norm and use this dataset instead to construct the complete binary tree. Thus the following corollary is immediate.

Corollary 25. *There is a data structure that can be prepared in $O(nD)$ time which enables generating a sample from $\kappa(\cdot)$ in $O(\log n)$ time.*

Looking ahead, the purpose of preprocessing is to enable fast sampling from the distribution $\kappa(\cdot|C)$ used in $\text{SAMPLE}(\mathcal{L}, C, m)$, Algorithm 2.

C.2. ANNS

In order to employ the rejection sampling idea, the probability of accepting a sampled point $x \in \mathcal{X}$ needs to be proportional to $\text{cost}(x, S)$, where S is the current set of centers. Again, computing this quantity takes $\Theta(kD)$ time, which contributes $\Theta(k^2D)$ time on performing this sampling k times. Thus, we apply another optimization technique; we use an approximate nearest neighbor data structure to approximate the distance between x and the closest center.

In particular, we utilize the following data structure, developed by (Cohen-Addad et al., 2020a), which is based on locality-sensitive hash functions (Andoni & Indyk, 2008).

Lemma 26. (LSH-based ANNS data structure) *For any set of points $P = \{p_1, \dots, p_t\} \subset \mathbb{R}^D$ and parameter $\varrho < 1$, there exists a data structure $\mathcal{L}$ with operations **Insert** and **Query** such that with probability at least $1 - 1/t$, the following guarantees hold⁷*

1. **Insert** $(\mathcal{L}, p)$ inserts the point $p \in P$ into $\mathcal{L}$ in time $O(D \log \eta(t \log \eta)^\varrho)$.
2. **Query** $(\mathcal{L}, p)$ returns a point such that $\|p - \text{Query}(\mathcal{L}, p)\| \leq \frac{1}{\sqrt{\varrho}} \cdot \min_{q \in \mathcal{L}} \|p - q\|$ and the query time is $O(D \log \eta(t \log \eta)^\varrho)$.
3. $\mathcal{L}$ is monotone under insertions: the distance between p and **Query** $(\mathcal{L}, p)$ is non-increasing after inserting more points.

Here, η is the aspect ratio of the set P defined as $\eta(P) = \frac{\max_{x \neq y \in P} \|x - y\|}{\min_{x \neq y \in P} \|x - y\|}$.

We shall be using this data structure $\mathcal{L}$ to store the sampled centers S and to query the approximate nearest center from S . To this end, we define $\text{cost}^{\mathcal{L}}(P, S) = \sum_{p \in P} \|p - \text{Query}(\mathcal{L}, p)\|^2$ and $\pi^{\mathcal{L}}(p|S) = \frac{\text{cost}^{\mathcal{L}}(p, S)}{\text{cost}^{\mathcal{L}}(P, S)}$ for a set of points P and a set of centers $S \subset \mathbb{R}^D$. Conditioned on the data structure being successful we have the following :

Lemma 27 (LSH approximation bounds). *Let $X = \{x_1, \dots, x_n\} \subset \mathbb{R}^D$ be a dataset and $S \subset X$ be a set of centers sampled from X . Suppose that the data structure $\mathcal{L}$ with parameter $\varrho < 1$ is successful and each $c \in S$ is inserted into the data structure $\mathcal{L}$. Then for each $x \in X$, the following holds*

$$\text{cost}(x, S) \leq \text{cost}^{\mathcal{L}}(x, S) \leq \varrho^{-1} \cdot \text{cost}(x, S)$$

from which we also have

$$\varrho \cdot \pi(x|S) \leq \pi^{\mathcal{L}}(x|S) \leq \varrho^{-1} \cdot \pi(x|S)$$

Remark 28 (Failure Probability of the LSH data structure). *For the rest of the analysis, we condition on the event that the data structure is successful. To understand the effect of the failure probability of the LSh data structure on the approximation guarantee, we refer the reader to a similar discussion from (Cohen-Addad et al., 2020b).*

Given the dataset, the hash tables which constitute the LSH data structure are chosen in the beginning in such a manner that with probability $1 - 1/n$, they follow certain desirable property (farther pairs of points map to distinct hashes, closer points map to same hash). See Corollary D.6 of previous work (Cohen-Addad et al., 2020b).

Assuming this property successfully holds for all $\binom{n}{2}$ points in the dataset, the Insert and Query operations will always be successful. So the failure probability of $1/n$ is valid only during the initialization phase; henceforth we can work with the assumption that all operations are successful with probability $1 - 1/n$. Also, see the footnote on Page 6 of previous work (Cohen-Addad et al., 2020b).

Suppose we repeat our LSH initialization followed by the seeding procedure M times and take the best clustering. Then with probability atleast $1 - p = 1 - 1/n^M$, one of the initializations would be successful. In that case, the expected cost is bounded by $O(\log k) \text{opt}_k$. In the case of failure, since we choose the first center uniformly at random, the expected cost is bounded by 2opt_1 . Hence, the total expected cost can be bounded as : $\mathbb{E}[\text{ALG}] \leq (1 - p) \cdot O(\log k) \text{opt}_k + 2p \cdot \text{opt}_1 \leq O(\log k) \cdot \text{opt}_k (1 + 2p\beta_k / \log k)$.

Choosing $M = \log_n(2n\beta_k)$ shows that $\mathbb{E}[\text{ALG}] \leq O(\log k) \text{opt}_k (1 + O(1/n))$. Hence, the effect of LSH failure has negligible impact on the output quality of our method.

C.3. Analysis of (ϱ, δ) - k -means++

The speedup in our algorithm can be attributed to two things; the first being using rejection sampling for sampling for the D^2 distribution and second, using the approximate nearest center for fast computation of the rejection sampling probability. Our goal is to bound the expected cost of the output of our algorithm. To do this, we analyze the following *abstract* version of k -means++, which we call (ϱ, δ) - k -means++.

Algorithm 6 (ϱ, δ) - k -means++

Input: dataset $\mathcal{X} = \{x_1, \dots, x_n\} \subset \mathbb{R}^D$, parameters $0 < \varrho, \delta < 1$ and $k \in \mathbb{N}$

- 1: $c_1 \leftarrow \text{Uniform}(\mathcal{X})$
 - 2: $\mathcal{L} \leftarrow$ ANNS data structure with parameter ϱ
 - 3: $\text{Insert}(c_1, \mathcal{L})$; $C \leftarrow \{c_1\}$
 - 4: **for** $i = 2$ to k **do**
 - 5: Sample y with probability $\pi_{(\varrho, \delta)}(y|C) := (1 - \delta) \cdot \pi^{\mathcal{L}}(y|C) + \delta \cdot \frac{1}{n}$
 - 6: $c_i \leftarrow y$; $\text{Insert}(c_i, \mathcal{L})$; $C \leftarrow C \cup \{c_i\}$
 - 7: **end for**
 - 8: **Output** C
-

C.3.1. THE PERTURBED DISTRIBUTION

Definition 29 ((ϱ, δ) perturbed D^2 distribution). *Let $\mathcal{X} = \{x_1, \dots, x_n\} \subset \mathbb{R}^d$ be a dataset and $S \subset X$ be a set of centers sampled from X . Suppose that the data structure $\mathcal{L}$ with parameter $\varrho < 1$ is successful and each $c \in S$ is inserted into the*

data structure $\mathcal{L}$. Then for each $x \in \mathcal{X}$, we define the (ϱ, δ) perturbed D^2 distribution as :

$$\pi_{(\varrho, \delta)}(x|S) = (1 - \delta) \cdot \pi^{\mathcal{L}}(x|S) + \delta \cdot \frac{1}{n}$$

Now, we provide some useful bounds that a point is chosen according to the perturbed distribution, conditioned on the event that it belongs to a specified set of points beforehand. These bounds will be useful later in several places.

Lemma 30. (Bounds on perturbed sampling probabilities) Let $\mathcal{X} = \{x_1, \dots, x_n\} \subset \mathbb{R}^d$ be a dataset and $\mathcal{Y} \subseteq \mathcal{X}$ be an arbitrary non-empty subset of $\mathcal{X}$. Let $C \subset \mathcal{X}$ be a set of centers and suppose the data structure $\mathcal{L}$ with parameter $\varrho < 1$ is successful and each $c \in C$ is inserted into $\mathcal{L}$. For any point $y \in \mathcal{Y}$ and parameter $\delta \in (0, 1)$, the following hold:

1. $\Pr_{y \sim \pi_{(\varrho, \delta)}(\cdot|C)}[y \mid y \in \mathcal{Y}] \leq \frac{\text{cost}^{\mathcal{L}}(y, C)}{\text{cost}^{\mathcal{L}}(\mathcal{Y}, C)} + \frac{\delta}{1 - \delta} \cdot \frac{1}{n} \cdot \frac{\text{cost}^{\mathcal{L}}(\mathcal{X}, C)}{\text{cost}^{\mathcal{L}}(\mathcal{Y}, C)}$
2. $\Pr_{y \sim \pi_{(\varrho, \delta)}(\cdot|C)}[y \mid y \in \mathcal{Y}] \geq \frac{\text{cost}^{\mathcal{L}}(y, C)}{\text{cost}^{\mathcal{L}}(\mathcal{Y}, C)} - \frac{\delta}{1 - \delta} \cdot \frac{|\mathcal{Y}|}{n} \cdot \frac{\text{cost}^{\mathcal{L}}(y, C) \cdot \text{cost}^{\mathcal{L}}(\mathcal{X}, C)}{\text{cost}^{\mathcal{L}}(\mathcal{Y}, C)^2}$

Proof. Let us first compute the probability that a point sampled from the perturbed D^2 distribution belongs to $\mathcal{Y}$:

$$\Pr_{y \sim \pi_{(\varrho, \delta)}(\cdot|C)}[y \in \mathcal{Y}] = \sum_{y' \in \mathcal{Y}} \left((1 - \delta) \frac{\text{cost}^{\mathcal{L}}(y', C)}{\text{cost}^{\mathcal{L}}(\mathcal{X}, C)} + \delta \cdot \frac{1}{n} \right) = (1 - \delta) \frac{\text{cost}^{\mathcal{L}}(\mathcal{Y}, C)}{\text{cost}^{\mathcal{L}}(\mathcal{X}, C)} + \delta \cdot \frac{|\mathcal{Y}|}{n}$$

Hence the required conditional probability is

$$\Pr_{y \sim \pi_{(\varrho, \delta)}(\cdot|C)}[y \mid y \in \mathcal{Y}] = \frac{(1 - \delta) \frac{\text{cost}^{\mathcal{L}}(y, C)}{\text{cost}^{\mathcal{L}}(\mathcal{X}, C)} + \delta \cdot \frac{1}{n}}{(1 - \delta) \frac{\text{cost}^{\mathcal{L}}(\mathcal{Y}, C)}{\text{cost}^{\mathcal{L}}(\mathcal{X}, C)} + \delta \cdot \frac{|\mathcal{Y}|}{n}}$$

For part 1, we have:

$$\begin{aligned} \frac{(1 - \delta) \frac{\text{cost}^{\mathcal{L}}(y, C)}{\text{cost}^{\mathcal{L}}(\mathcal{X}, C)} + \delta \cdot \frac{1}{n}}{(1 - \delta) \frac{\text{cost}^{\mathcal{L}}(\mathcal{Y}, C)}{\text{cost}^{\mathcal{L}}(\mathcal{X}, C)} + \delta \cdot \frac{|\mathcal{Y}|}{n}} &\leq \frac{(1 - \delta) \frac{\text{cost}^{\mathcal{L}}(y, C)}{\text{cost}^{\mathcal{L}}(\mathcal{X}, C)} + \delta \cdot \frac{1}{n}}{(1 - \delta) \frac{\text{cost}^{\mathcal{L}}(\mathcal{Y}, C)}{\text{cost}^{\mathcal{L}}(\mathcal{X}, C)}} \\ &= \frac{\text{cost}^{\mathcal{L}}(y, C)}{\text{cost}^{\mathcal{L}}(\mathcal{Y}, C)} + \frac{\delta}{1 - \delta} \cdot \frac{1}{n} \cdot \frac{\text{cost}^{\mathcal{L}}(\mathcal{X}, C)}{\text{cost}^{\mathcal{L}}(\mathcal{Y}, C)} \end{aligned}$$

For part 2, we have:

$$\begin{aligned} \frac{(1 - \delta) \frac{\text{cost}^{\mathcal{L}}(y, C)}{\text{cost}^{\mathcal{L}}(\mathcal{X}, C)} + \delta \cdot \frac{1}{n}}{(1 - \delta) \frac{\text{cost}^{\mathcal{L}}(\mathcal{Y}, C)}{\text{cost}^{\mathcal{L}}(\mathcal{X}, C)} + \delta \cdot \frac{|\mathcal{Y}|}{n}} &= \frac{\text{cost}^{\mathcal{L}}(y, C)}{\text{cost}^{\mathcal{L}}(\mathcal{Y}, C)} \left(\frac{1 + \frac{\delta}{1 - \delta} \cdot \frac{\text{cost}^{\mathcal{L}}(\mathcal{X}, C)}{n \cdot \text{cost}^{\mathcal{L}}(y, C)}}{1 + \frac{\delta}{1 - \delta} \cdot \frac{|\mathcal{Y}| \cdot \text{cost}^{\mathcal{L}}(\mathcal{X}, C)}{n \cdot \text{cost}^{\mathcal{L}}(\mathcal{Y}, C)}} \right) \\ &\geq \frac{\text{cost}^{\mathcal{L}}(y, C)}{\text{cost}^{\mathcal{L}}(\mathcal{Y}, C)} \left(1 + \frac{\delta}{1 - \delta} \cdot \frac{|\mathcal{Y}| \cdot \text{cost}^{\mathcal{L}}(\mathcal{X}, C)}{n \cdot \text{cost}^{\mathcal{L}}(\mathcal{Y}, C)} \right)^{-1} \\ &\geq \frac{\text{cost}^{\mathcal{L}}(y, C)}{\text{cost}^{\mathcal{L}}(\mathcal{Y}, C)} \left(1 - \frac{\delta}{1 - \delta} \cdot \frac{|\mathcal{Y}| \cdot \text{cost}^{\mathcal{L}}(\mathcal{X}, C)}{n \cdot \text{cost}^{\mathcal{L}}(\mathcal{Y}, C)} \right) \end{aligned}$$

where in the last step we used $(1 + x)^{-1} \geq 1 - x$ for any $x \geq 0$. □

C.3.2. SOME USEFUL LEMMAS

Suppose $S = \{s_1, \dots, s_k\}$ is an optimal set of centers for the dataset $\mathcal{X}$ so that $\text{cost}(\mathcal{X}, S) = \text{opt}_k(\mathcal{X})$. Let $\mathcal{X}_j$ denote the set of points from $\mathcal{X}$ whose nearest center is s_j . This induces a partition $\mathcal{X} = \bigcup_{j \in [k]} \mathcal{X}_j$ where $\mathcal{X}_i \cap \mathcal{X}_j = \emptyset$ and $s_j = \mu(\mathcal{X}_j)$. Notice that we can decompose the optimal k -means cost as $\text{opt}_k(\mathcal{X}) = \sum_{j \in [k]} \text{opt}_1(\mathcal{X}_j)$. Let us state a few results from (Arthur & Vassilvitskii, 2007) that will be useful.

Lemma 31. (First center cost, Lemma 3.1 of (Arthur & Vassilvitskii, 2007)) Let c_1 be the first center chosen by Algorithm 6. For an optimal choice of cluster centers S , let $\mathcal{X}_j$ denote the cluster corresponding to the j th center s_j for $j \in [k]$. Then:

$$\mathbb{E}[\text{cost}(\mathcal{X}_j, c_1) | c_1 \in \mathcal{X}_j] = 2\text{opt}_1(\mathcal{X}_j)$$

Lemma 32 (Standard k -means++ bound (Lemma 3.2 of (Arthur & Vassilvitskii, 2007))). *For an optimal choice of cluster centers S , let $\mathcal{X}_j$ denote the cluster corresponding to the j th center for $j \in [k]$. Let C be a set of chosen centers, and let y be a new center chosen according to the (standard) D^2 distribution such that $y \in \mathcal{X}_j$. Then:*

$$\mathbf{E}[\text{cost}(\mathcal{X}_j, C \cup \{y\}) \mid C, \{y \in \mathcal{X}_j\}] \leq 8\text{opt}_1(\mathcal{X}_j)$$

The following is an analogous version of Lemma 3.2 of (Arthur & Vassilvitskii, 2007), but we need additional analysis since we use the perturbed distribution instead of the exact D^2 distribution:

Lemma 33 (Perturbed center selection bound). *Consider an iteration of Algorithm 6 after the first one and let C be the current set of centers. Let y be the center chosen in the current iteration. For an optimal choice of cluster centers S , let $\mathcal{X}_j$ denote the cluster corresponding to the j th center for $j \in [k]$. Then for any $C \neq \emptyset$:*

$$\mathbf{E}[\text{cost}(\mathcal{X}_j, C \cup \{y\}) \mid C, \{y \in \mathcal{X}_j\}] \leq \varrho^{-1} \left(8\text{opt}_1(\mathcal{X}_j) + \frac{\delta}{1-\delta} \frac{|\mathcal{X}_j|}{n} \text{cost}(\mathcal{X}, C) \right)$$

Proof. When the new center y is added, each point $x \in \mathcal{X}_j$ contributes $\text{cost}(x, C \cup \{y\}) = \min(\|x - y\|^2, \text{cost}(x, C))$ to the overall cost, where $\mathcal{L}$ now contains $C \cup \{y\}$. The expected cost of the cluster $\mathcal{X}_j$ can hence be written as:

$$\begin{aligned} & \mathbf{E}[\text{cost}(\mathcal{X}_j, C \cup \{y\}) \mid C, \{y \in \mathcal{X}_j\}] && \left. \vphantom{\mathbf{E}[\text{cost}(\mathcal{X}_j, C \cup \{y\}) \mid C, \{y \in \mathcal{X}_j\}]} \right\} \text{Expanding} \\ &= \sum_{y \in \mathcal{X}_j} \mathbf{Pr}_{y \sim \pi_{(\varrho, \delta)}(\cdot \mid C)}[y \mid y \in \mathcal{X}_j] \cdot \text{cost}(\mathcal{X}_j, C \cup \{y\}) && \left. \vphantom{\sum_{y \in \mathcal{X}_j} \mathbf{Pr}_{y \sim \pi_{(\varrho, \delta)}(\cdot \mid C)}[y \mid y \in \mathcal{X}_j] \cdot \text{cost}(\mathcal{X}_j, C \cup \{y\})]} \right\} \text{Expanding} \\ &= \sum_{y \in \mathcal{X}_j} \mathbf{Pr}_{y \sim \pi_{(\varrho, \delta)}(\cdot \mid C)}[y \mid y \in \mathcal{X}_j] \cdot \left(\sum_{x \in \mathcal{X}_j} \text{cost}(x, C \cup \{y\}) \right) && \left. \vphantom{\sum_{y \in \mathcal{X}_j} \mathbf{Pr}_{y \sim \pi_{(\varrho, \delta)}(\cdot \mid C)}[y \mid y \in \mathcal{X}_j] \cdot \left(\sum_{x \in \mathcal{X}_j} \text{cost}(x, C \cup \{y\}) \right)} \right\} \text{Lemma 30 item (I)} \\ &\leq \sum_{y \in \mathcal{X}_j} \left(\frac{\text{cost}^{\mathcal{L}}(y, C)}{\text{cost}^{\mathcal{L}}(\mathcal{X}_j, C)} + \frac{\delta}{1-\delta} \frac{1}{n} \frac{\text{cost}^{\mathcal{L}}(\mathcal{X}, C)}{\text{cost}^{\mathcal{L}}(\mathcal{X}_j, C)} \right) \cdot \left(\sum_{x \in \mathcal{X}_j} \text{cost}(x, C \cup \{y\}) \right) \\ &\leq \underbrace{\sum_{y \in \mathcal{X}_j} \frac{\text{cost}^{\mathcal{L}}(y, C)}{\text{cost}^{\mathcal{L}}(\mathcal{X}_j, C)} \sum_{x \in \mathcal{X}_j} \min(\|x - y\|^2, \text{cost}(y, C))}_{(I)} \\ &\quad + \underbrace{\sum_{y \in \mathcal{X}_j} \left(\frac{\delta}{1-\delta} \frac{1}{n} \frac{\text{cost}(\mathcal{X}, C)}{\text{cost}^{\mathcal{L}}(\mathcal{X}_j, C)} \right) \sum_{x \in \mathcal{X}_j} \min(\|x - y\|^2, \text{cost}(x, C))}_{(II)} \end{aligned}$$

where in the third step we used part 1 of Lemma 30. Using Lemma 27 and Lemma 3.2 of (Arthur & Vassilvitskii, 2007), we have for (I) :

$$\sum_{y \in \mathcal{X}_j} \frac{\text{cost}^{\mathcal{L}}(y, C)}{\text{cost}^{\mathcal{L}}(\mathcal{X}_j, C)} \sum_{x \in \mathcal{X}_j} \min(\|x - y\|^2, \text{cost}(x, C)) \leq 8\varrho^{-1}\text{opt}_1(\mathcal{X}_j)$$

For the second term (II), noting that $\sum_{x \in \mathcal{X}_j} \min(\|x - y\|^2, \text{cost}(x, C)) \leq \sum_{x \in \mathcal{X}_j} \text{cost}(x, C) = \text{cost}(\mathcal{X}_j, C)$, we have:

$$\begin{aligned} & \sum_{y \in \mathcal{X}_j} \left(\frac{\delta}{1-\delta} \frac{1}{n} \frac{\text{cost}^{\mathcal{L}}(\mathcal{X}, C)}{\text{cost}^{\mathcal{L}}(\mathcal{X}_j, C)} \right) \sum_{x \in \mathcal{X}_j} \min(\|x - y\|^2, \text{cost}(x, C)) \\ & \leq \varrho^{-1} \cdot \sum_{y \in \mathcal{X}_j} \left(\frac{\delta}{1-\delta} \frac{1}{n} \frac{\text{cost}(\mathcal{X}, C)}{\text{cost}(\mathcal{X}_j, C)} \right) \text{cost}(\mathcal{X}_j, C) = \frac{\varrho^{-1}\delta}{1-\delta} \frac{|\mathcal{X}_j|}{n} \text{cost}(\mathcal{X}, C) \end{aligned}$$

Combining both terms completes the proof. $\square$

C.3.3. POTENTIAL ANALYSIS

Following (Dasgupta, 2003), we first setup some notation. Let $t \in \{0, \dots, k\}$ denote the number of centers already chosen by Algorithm 6. Let $C_t := \{c_1, \dots, c_t\}$ be the set of centers after t iterations. We say that an optimal cluster $\mathcal{X}_j$ is *covered* by C_t if at least one of the chosen centers is in $\mathcal{X}_j$. If not, then it is *uncovered*. We denote $H_t = \{j \in \{1, \dots, k\} : \mathcal{X}_j \cap C_t \neq \emptyset\}$ and $U_t = \{1, \dots, k\} \setminus H_t$. Similarly, the dataset $\mathcal{X}$ can be partitioned into two parts: $\mathcal{H}_t \subset \mathcal{X}$ being the points belonging to *covered* clusters and $\mathcal{U}_t = \mathcal{X} \setminus \mathcal{H}_t$ being the points belonging to *uncovered* clusters. Let $W_t = t - |H_t|$ denote the number of *wasted* iterations so far, i.e., the number of iterations in which no new cluster was covered. Note that we always have $|H_t| \leq t$ and hence $|U_t| \geq k - t$. For any $\mathcal{Y} \subset \mathcal{X}$, we use the notation $\text{cost}_t(\mathcal{Y}) = \text{cost}(\mathcal{Y}, C_t)$ and $\text{cost}_t^{\mathcal{L}}(\mathcal{Y}) = \text{cost}^{\mathcal{L}}(\mathcal{Y}, C_t)$ for brevity. The total cost can be decomposed as:

$$\text{cost}_t(\mathcal{X}) = \text{cost}_t(\mathcal{H}_t) + \text{cost}_t(\mathcal{U}_t) \leq \text{cost}_t(\mathcal{H}_t) + \text{cost}_t^{\mathcal{L}}(\mathcal{U}_t)$$

We can bound the first term directly using the previous lemmas.

Lemma 34. (Cost bound for covered clusters) *For each $t \in \{1, \dots, k\}$ the following holds:*

$$\mathbf{E}[\text{cost}_t(\mathcal{H}_t)] \leq \varrho^{-1} \left(8\text{opt}_k(\mathcal{X}) + \frac{2\delta}{1-\delta} \text{opt}_1(\mathcal{X}) \right)$$

Proof. Consider any cluster $j \in H_t$. This means that at some iteration t_j , a point $y \in \mathcal{X}_j$ would have been selected as the new center c_{t_j} so that j becomes covered. This observation allows us to use Lemma 33 in the following manner :

$$\begin{aligned} \mathbf{E}[\text{cost}_t(\mathcal{H}_t)] &= \mathbf{E} \left[\sum_{j \in H_t} \text{cost}_t(\mathcal{X}_j) \right] && \left. \begin{array}{l} \text{Using linearity of} \\ \text{expectation} \end{array} \right\} \\ &= \sum_{j \in H_t} \mathbf{E}[\text{cost}_t(\mathcal{X}_j)] && \left. \begin{array}{l} \text{Using Lemma 33} \end{array} \right\} \\ &\leq \sum_{j \in H_t} \varrho^{-1} \left(8\text{opt}_1(\mathcal{X}_j) + \frac{\delta}{1-\delta} \frac{|\mathcal{X}_j|}{n} \text{cost}(\mathcal{X}, C_{t_j-1}) \right) && \left. \begin{array}{l} \text{Using} \\ \mathbf{E}[\text{cost}(\mathcal{X}, C_{t_j-1})] \leq \\ \mathbf{E}[\text{cost}(\mathcal{X}, c_1)] = \\ 2\text{opt}_1(\mathcal{X}) \end{array} \right\} \\ &\leq \sum_{j \in H_t} \varrho^{-1} \left(8\text{opt}_1(\mathcal{X}_j) + \frac{\delta}{1-\delta} \frac{|\mathcal{X}_j|}{n} \cdot 2\text{opt}_1(\mathcal{X}) \right) && \left. \begin{array}{l} \text{Expanding the sum} \\ \text{over } j \in H_t \text{ to} \\ j \in [k] \end{array} \right\} \\ &= \varrho^{-1} \left(8 \sum_{j \in [k]} \text{opt}_1(\mathcal{X}_j) + \frac{2\delta}{1-\delta} \cdot \frac{\sum_{j \in [k]} |\mathcal{X}_j|}{n} \cdot \text{opt}_1(\mathcal{X}) \right) && \left. \begin{array}{l} \text{Using } \text{opt}_k(\mathcal{X}) = \\ \sum_{j \in [k]} \text{opt}_1(\mathcal{X}_j) \end{array} \right\} \\ &= \varrho^{-1} \left(8\text{opt}_k(\mathcal{X}) + \frac{2\delta}{1-\delta} \text{opt}_1(\mathcal{X}) \right) \end{aligned}$$

This completes the proof. □

Potential function. To bound the second term, i.e., the cost of the uncovered clusters, we use the potential function method introduced in (Dasgupta, 2013), with a slightly modified potential function:

$$\varphi_t = \frac{W_t}{|U_t|} \text{cost}_t^{\mathcal{L}}(\mathcal{U}_t)$$

Instead of *paying* the complete clustering cost of the uncovered clusters at once, we make sure that at the end of iteration t , we have paid an amount of at least φ_t . Observe that when $t = k$, we have $W_k = |U_k|$, so the potential becomes $\text{cost}_k^{\mathcal{L}}(\mathcal{U}_k)$, which is atleast $\text{cost}_k(\mathcal{U}_k)$, i.e. the total cost of the uncovered clusters returned by Algorithm 3. We now show how to bound the expected increase in the potential, i.e., $\varphi_{t+1} - \varphi_t$ at each iteration. The total potential bound will then follow by expressing φ_k as a telescoping sum of the increments.

Suppose t centers have been chosen. The next center c_{t+1} is chosen which belongs to some optimal cluster $\mathcal{X}_j$. We consider two cases: the first case is when $j \in U_t$, i.e., a new cluster is covered, and the second case is when $j \in H_t$, i.e., the center is chosen from an already covered cluster. We shall denote the set of all random variables after the end of iteration t by $\mathbf{F}_t$.

Lemma 35. (Potential increment for uncovered clusters) For any $t \in \{1, \dots, k-1\}$, the following holds:

$$\mathbf{E}[\varphi_{t+1} - \varphi_t \mid \mathbf{F}_t, \{j \in U_t\}] \leq \frac{2\delta}{1-\delta} \frac{t}{\max(1, k-t-1)^2} \text{opt}_1(\mathcal{X})$$

Proof. When $j \in U_t$, we have $W_{t+1} = W_t$, $H_{t+1} = H_t \cup \{j\}$, and $U_{t+1} = U_t \setminus \{j\}$. Thus using the monotonicity of the data structure L , we have

$$\varphi_{t+1} = \frac{W_{t+1}}{|U_{t+1}|} \text{cost}_{t+1}^{\mathcal{L}}(\mathcal{U}_{t+1}) \leq \frac{W_t}{|U_t| - 1} (\text{cost}_t^{\mathcal{L}}(\mathcal{U}_t) - \text{cost}_t^{\mathcal{L}}(\mathcal{X}_j))$$

We can use part 2 of Lemma 30 for getting a lower bound on the second term:

$$\begin{aligned} & \mathbf{E}[\text{cost}_t^{\mathcal{L}}(\mathcal{X}_j) \mid \mathbf{F}_t, \{j \in U_t\}] \\ & \geq \sum_{j \in U_t} \left(\frac{\text{cost}_t^{\mathcal{L}}(\mathcal{X}_j)}{\text{cost}_t^{\mathcal{L}}(\mathcal{U}_t)} - \frac{\delta}{1-\delta} \frac{|\mathcal{X}_j|}{n} \frac{\text{cost}_t^{\mathcal{L}}(\mathcal{X}_j) \text{cost}_t^{\mathcal{L}}(\mathcal{X})}{\text{cost}_t^{\mathcal{L}}(\mathcal{U}_t)^2} \right) \text{cost}_t^{\mathcal{L}}(\mathcal{X}_j) \\ & \geq \left(1 - \frac{\delta}{1-\delta} \frac{\text{cost}_t^{\mathcal{L}}(\mathcal{X})}{\text{cost}_t^{\mathcal{L}}(\mathcal{U}_t)} \right) \sum_{j \in U_t} \frac{\text{cost}_t^{\mathcal{L}}(\mathcal{X}_j)^2}{\text{cost}_t^{\mathcal{L}}(\mathcal{U}_t)} \end{aligned}$$

where in the second step we used the fact that $|\mathcal{X}_j| \leq n$ for each $j \in U_t$. We can use the Cauchy-Schwarz inequality⁸ to simplify the last expression as follows:

$$|U_t|^2 \sum_{j \in U_t} \text{cost}_t^{\mathcal{L}}(\mathcal{X}_j)^2 \geq |U_t| \sum_{j \in U_t} \text{cost}_t^{\mathcal{L}}(\mathcal{X}_j) = |U_t| \text{cost}_t^{\mathcal{L}}(\mathcal{U}_t)$$

This shows that

$$\mathbf{E}[\text{cost}_t^{\mathcal{L}}(\mathcal{X}_j) \mid \mathbf{F}_t, \{j \in U_t\}] \geq \frac{\text{cost}_t^{\mathcal{L}}(\mathcal{U}_t)}{|U_t|} - \frac{\delta}{1-\delta} \frac{\text{cost}_t^{\mathcal{L}}(\mathcal{X})}{|U_t|}$$

Now,

$$\begin{aligned} \mathbf{E}[\varphi_{t+1} \mid \mathbf{F}_t, i \in U_t] & \leq \frac{W_t}{|U_t| - 1} (\text{cost}_t^{\mathcal{L}}(\mathcal{U}_t) - \mathbf{E}[\text{cost}_t^{\mathcal{L}}(\mathcal{X}_j) \mid \mathbf{F}_t, i \in U_t]) \\ & \leq \frac{W_t}{|U_t| - 1} \left(\text{cost}_t^{\mathcal{L}}(\mathcal{U}_t) - \frac{\text{cost}_t^{\mathcal{L}}(\mathcal{U}_t)}{|U_t|} + \frac{\delta}{1-\delta} \frac{\text{cost}_t^{\mathcal{L}}(\mathcal{X})}{|U_t|} \right) \\ & = \varphi_t + \frac{\delta}{1-\delta} \frac{W_t}{|U_t| (|U_t| - 1)} \text{cost}_t^{\mathcal{L}}(\mathcal{X}) \end{aligned}$$

Recall that $W_t \leq t$ and $|U_t| \geq k-t$. So for $t \leq k-2$, the following holds after taking expectation:

$$\begin{aligned} \mathbf{E}[\varphi_{t+1} - \varphi_t \mid \mathbf{F}_t, \{j \in U_t\}] & \leq \frac{\delta}{1-\delta} \frac{t}{(k-t-1)^2} \mathbf{E}[\text{cost}_t^{\mathcal{L}}(\mathcal{X}) \mid \mathbf{F}_t, j \in U_t] \\ & \leq \frac{2\delta}{1-\delta} \frac{t}{(k-t-1)^2} \text{opt}_1(\mathcal{X}) \end{aligned}$$

Now consider the case when $t = k-1$. We cannot use the above argument directly because it may so happen that $|U_{k-1}| = 1$. If this happens, the potential of the uncovered clusters is always 0. This only happens when a new cluster is covered in each iteration. Let this event be A (for *All Clusters* being covered). Denoting $\mathbf{E}$ to be the event: $\{\mathbf{F}_{k-1}, \{j \in U_{k-1}\}\}$, we have the following:

$$\begin{aligned} \mathbf{E}[\varphi_k - \varphi_{k-1} \mid \mathbf{E}] & = \mathbf{E}[\varphi_k - \varphi_{k-1} \mid \mathbf{E}, A] \Pr[A \mid \mathbf{E}] + \mathbf{E}[\varphi_k - \varphi_{k-1} \mid \mathbf{E}, \neg A] \Pr[\neg A \mid \mathbf{E}] \quad \left. \begin{array}{l} \text{Using} \\ \mathbf{E}[\varphi_k - \varphi_{k-1} \mid \mathbf{E}, A] \leq 0 \end{array} \right\} \\ & \leq \mathbf{E}[\varphi_k - \varphi_{k-1} \mid \mathbf{E}, \neg A] \Pr[\neg A \mid \mathbf{E}] \\ & \leq \mathbf{E}[\varphi_k - \varphi_{k-1} \mid \mathbf{E}, \neg A] \\ & \leq \frac{2\delta t}{1-\delta} \text{opt}_1(\mathcal{X}) \end{aligned}$$

where in the last line we used the fact that $|U_{k-1}| > 1$ if all clusters are not covered. We also used the fact that $\mathbf{E}[\varphi_k - \varphi_{k-1} \mid \mathbf{E}, A] \leq 0$ since in this case there are no uncovered clusters. Combining both cases completes the proof. $\square$

⁸For lists of numbers $a_1, \dots, a_m$ and $b_1, \dots, b_m$ we have $(\sum_i a_i b_i)^2 \leq (\sum_i a_i^2) (\sum_i b_i^2)$.

Lemma 36. (Potential increment for covered clusters) *For any $t \in \{1, \dots, k-1\}$, the following holds:*

$$\mathbf{E}[\varphi_{t+1} - \varphi_t \mid \mathbf{F}_t, \{i \in H_t\}] \leq \frac{\text{cost}_t^{\mathcal{L}}(\mathcal{U}_t)}{k-t}$$

Proof. When $i \in H_t$, we have $H_{t+1} = H_t$, $W_{t+1} = W_t + 1$, and $U_{t+1} = U_t$. Thus,

$$\begin{aligned} \varphi_{t+1} - \varphi_t &= \frac{W_{t+1}}{|U_{t+1}|} \text{cost}_{t+1}^{\mathcal{L}}(\mathcal{U}_{t+1}) - \frac{W_t}{|U_t|} \text{cost}_t^{\mathcal{L}}(\mathcal{U}_t) \\ &\leq \frac{W_t + 1}{|U_t|} \text{cost}_t^{\mathcal{L}}(\mathcal{U}_t) - \frac{W_t}{|U_t|} \text{cost}_t^{\mathcal{L}}(\mathcal{U}_t) \\ &= \frac{\text{cost}_t^{\mathcal{L}}(\mathcal{U}_t)}{|U_t|} \leq \frac{\text{cost}_t^{\mathcal{L}}(\mathcal{U}_t)}{k-t} \end{aligned}$$

□

We can now combine the two cases to get:

Lemma 37 (Combined potential increment bound). *For any $t \in \{1, \dots, k-1\}$, the following holds: $\mathbf{E}[\varphi_{t+1} - \varphi_t \mid \mathbf{F}_t] \leq (1 - \delta) \frac{\mathbf{E}[\text{cost}_t^{\mathcal{L}}(\mathcal{H}_t)]}{k-t} + \delta \left(\frac{2}{k-t} + \frac{2t}{\max(1, k-t-1)^2} \right) \text{opt}_1(\mathcal{X})$*

Proof. To compute the overall expectation, we have:

$$\mathbf{E}[\varphi_{t+1} - \varphi_t \mid \mathbf{F}_t] = \mathbf{E}[\varphi_{t+1} - \varphi_t \mid \mathbf{F}_t, \{i \in U_t\}] \Pr[i \in U_t] + \mathbf{E}[\varphi_{t+1} - \varphi_t \mid \mathbf{F}_t, \{i \in H_t\}] \Pr[i \in H_t]$$

We can bound the first term using Lemma 35:

$$\begin{aligned} &\mathbf{E}[\varphi_{t+1} - \varphi_t \mid \mathbf{F}_t, i \in U_t] \Pr[i \in U_t] \\ &\leq \mathbf{E}[\varphi_{t+1} - \varphi_t \mid \mathbf{F}_t, i \in U_t] \\ &\leq \frac{2\delta}{1-\delta} \frac{t}{\max(1, k-t-1)^2} \text{opt}_1(\mathcal{X}) \end{aligned}$$

and the second term using Lemma 36:

$$\begin{aligned} &\mathbf{E}[\varphi_{t+1} - \varphi_t \mid \mathbf{F}_t, i \in H_t] \Pr[i \in H_t] \\ &\leq \frac{\text{cost}_t^{\mathcal{L}}(\mathcal{U}_t)}{k-t} \left((1-\delta) \frac{\text{cost}_t^{\mathcal{L}}(\mathcal{H}_t)}{\text{cost}_t^{\mathcal{L}}(\mathcal{X})} + \delta \frac{|\mathcal{H}_t|}{n} \right) \\ &\leq (1-\delta) \frac{\text{cost}_t^{\mathcal{L}}(\mathcal{H}_t)}{k-t} + \delta \frac{\text{cost}_t^{\mathcal{L}}(\mathcal{X})}{k-t} \end{aligned}$$

where in the last step we used $\text{cost}_t^{\mathcal{L}}(\mathcal{U}_t) \leq \text{cost}_t^{\mathcal{L}}(\mathcal{X})$ and $|\mathcal{H}_t| \leq n$. Combining both terms completes the proof. □

C.3.4. FINAL APPROXIMATION GUARANTEE

We are now ready to provide a proof for the main theorem regarding (ϱ, δ) - k -means++:

Theorem 38 (Main approximation guarantee for (ϱ, δ) - k -means++). *Let $\mathcal{X} \subset \mathbb{R}^D$ be any dataset which is to be partitioned into k clusters. Let C be the set of centers returned by Algorithm 6 for any $\delta \in (0, 0.5)$ and $\varrho < 1$. The following approximation guarantee holds:*

$$\mathbf{E}[\text{cost}(\mathcal{X}, C)] \in O(\varrho^{-2} \log k) \text{opt}_k(\mathcal{X}) + \delta \cdot O(k + \varrho^{-2} \log k) \text{opt}_1(\mathcal{X})$$

Proof. At the end of k iterations, we have $\text{cost}(\mathcal{X}, C) \leq \text{cost}_k(\mathcal{H}_k) + \text{cost}_k^{\mathcal{L}}(\mathcal{U}_k) = \text{cost}_k(\mathcal{H}_k) + \varphi_k$. The first term can be bounded using Lemma 34. For the second term, we can express φ_k as a telescopic sum:

$$\begin{aligned}
 \mathbf{E}[\text{cost}(\mathcal{X}, C)] &\leq \mathbf{E}[\text{cost}_k(\mathcal{H}_k)] + \sum_{t=0}^{k-1} \mathbf{E}[\varphi_{t+1} - \varphi_t \mid \mathbf{F}_t] \\
 &\leq \mathbf{E}[\text{cost}_k(\mathcal{H}_k)] + \sum_{t=0}^{k-1} (1-\delta) \frac{\mathbf{E}[\text{cost}_t^{\mathcal{L}}(\mathcal{H}_t)]}{k-t} + \sum_{t=0}^{k-1} \delta \left(\frac{2}{k-t} + \frac{2t}{(1-\delta) \max(1, k-t-1)^2} \right) \text{opt}_1(\mathcal{X}) \\
 &\leq \mathbf{E}[\text{cost}_k(\mathcal{H}_k)] + \sum_{t=0}^{k-1} \frac{(1-\delta)}{\varepsilon} \frac{\mathbf{E}[\text{cost}_t(\mathcal{H}_t)]}{k-t} + \sum_{t=0}^{k-1} \delta \left(\frac{2}{k-t} + \frac{2t}{(1-\delta) \max(1, k-t-1)^2} \right) \text{opt}_1(\mathcal{X}) \\
 &\leq \varrho^{-1} \left(8\text{opt}_k(\mathcal{X}) + \frac{2\delta}{1-\delta} \text{opt}_1(\mathcal{X}) \right) \left(1 + (1-\delta) \cdot \varepsilon^{-1} \cdot \sum_{t=0}^{k-1} \frac{1}{k-t} \right) \\
 &\quad + \frac{2\delta}{1-\delta} \text{opt}_1(\mathcal{X}) \left(k + (1-\delta)(1 + \ln k) + \sum_{t=0}^{k-2} \frac{t}{(k-t-1)^2} \right)
 \end{aligned}$$

To simplify this, note that $\sum_{t=0}^{k-1} \frac{1}{k-t} \leq 1 + \ln k$ and $\sum_{t=0}^{k-2} \frac{t}{(k-t-1)^2} \leq k \sum_{t=1}^{\infty} t^{-2} = k\pi^2/6$.

$$\begin{aligned}
 \mathbf{E}[\text{cost}(\mathcal{X}, C)] &\leq \varrho^{-1} \left(8\text{opt}_k(\mathcal{X}) + \frac{2\delta}{1-\delta} \text{opt}_1(\mathcal{X}) \right) (1 + \varepsilon^{-1}(1 + \ln k)) + 2\delta \text{opt}_1(\mathcal{X}) (k + 2 + 2 \ln k + k\pi^2/3)
 \end{aligned}$$

For sufficiently large k , we obtain :

$$\mathbf{E}[\text{cost}(\mathcal{X}, C)] \in O(\varrho^{-2} \log k) \text{opt}_k(\mathcal{X}) + \frac{\delta}{1-\delta} \cdot O(k + \varrho^{-2} \log k) \text{opt}_1(\mathcal{X})$$

This completes the proof of the theorem. $\square$

C.4. Rejection Sampling

The naive implementation of k -means++ seeding involves iteratively selecting each cluster center using the D^2 distribution with respect to the current set of centers. Sampling a point from this distribution naively is expensive as it takes $O(nkd)$ time. To obtain a faster implementation, we sample each center from a distribution that *oversamples* the D^2 distribution (but is faster to sample from), and then perform rejection sampling to ensure that the overall distribution remains close to D^2 . Let us formally define what we mean by *oversampling* a distribution.

Definition 39. (τ -oversampling) Suppose π, κ define probability distributions over $\mathcal{X}$. The distribution κ is said to τ -oversample π for $\tau > 0$ if $\pi(x) \leq \tau \cdot \kappa(x)$ for each $x \in \mathcal{X}$.

Let π, κ be probability distributions over $\mathcal{X}$ such that κ τ -oversamples π . Suppose we have a collection of samples $x_1, x_2, \dots$ from the distribution D_2 . Consider the following strategy RejectionSample: iterate through samples $\{x_i\}_i$ and terminate if a sample x_i is accepted with probability $\varrho(x_i) = \frac{\pi(x_i)}{\tau \kappa(x_i)}$.

It is not difficult to argue that an accepted sample comes from the distribution π . Moreover, for any $\varepsilon \in (0, 1)$ it takes at most $\tau \ln(1/\varepsilon)$ samples from κ to accept a sample with probability at least $1 - \varepsilon$. Note that the above strategy may not need to know τ in advance (indeed, computing τ may be non-trivial), but only requires the ability to compute the quantity $\varrho(x_i)$. In the current form, our strategy does not have any control over the number of samples from κ which it may need to examine. However, a bound on the number of samples to be examined can be obtained if we are content with sampling from a slightly perturbed distribution. Suppose we have another distribution ν over $\mathcal{X}$. This time we are allowed to use samples coming from κ and ν and instead of a sample from π , we are content with obtaining a sample generated by a hybrid distribution $\pi_\delta(x) = (1 - \delta)\pi(x) + \delta\nu(x)$ for some small enough $\delta \in (0, 1)$. For this we can modify the above strategy which we now call RejectionSample(m): Iterate through m samples $x_1, \dots, x_m$ from κ , terminate if a sample x_i is accepted with

probability $\varrho(x_i)$. If no sample is accepted, terminate with a sample from ν . It can be shown that the *failure* probability δ diminishes with increasing m . Indeed, the following holds:

Lemma 40. (*Failure probability bound*) *The failure probability of $\text{RejectionSample}(m)$ satisfies $\delta \leq e^{-m/\tau}$.*

In our algorithm, we appropriately choose the distributions κ and ν over $\mathcal{X}$ so that κ τ -oversamples π (for a suitable τ) and for which we obtain samples efficiently.

C.5. Proof of Correctness for Rejection Sampling

We now prove that the rejection sampling procedure indeed produces samples from the desired distribution.

Lemma 41. (*Rejection sampling correctness*) *Let π, κ be probability distributions over $\mathcal{X}$ such that κ τ -oversamples π . If we sample x from κ and accept it with probability $\varrho(x) = \frac{\pi(x)}{\tau\kappa(x)}$, then the accepted sample comes from distribution π .*

Proof. For any $x \in \mathcal{X}$, the probability that x is the accepted sample is:

$$\begin{aligned} \Pr[\text{accept } x] &= \Pr[\text{sample } x \text{ from } \kappa] \cdot \Pr[\text{accept} \mid \text{sampld } x] \\ &= \kappa(x) \cdot \frac{\pi(x)}{\tau\kappa(x)} = \frac{\pi(x)}{\tau} \end{aligned}$$

The total probability of accepting any sample is:

$$\Pr[\text{accept}] = \sum_{x \in \mathcal{X}} \Pr[\text{accept } x] = \sum_{x \in \mathcal{X}} \frac{\pi(x)}{\tau} = \frac{1}{\tau}$$

Therefore, the conditional probability that the accepted sample is x is:

$$\Pr[x \mid \text{accept}] = \frac{\Pr[\text{accept } x]}{\Pr[\text{accept}]} = \frac{\pi(x)/\tau}{1/\tau} = \pi(x)$$

Thus, accepted samples follow distribution π . □

Lemma 42. (*Expected number of samples*) *Using the rejection sampling procedure with κ τ -oversampling π , the expected number of samples from κ needed to obtain one accepted sample is exactly τ .*

Proof. Each sample from κ is accepted with probability $1/\tau$. Therefore, the number of samples follows a geometric distribution with success probability $p = 1/\tau$, which has expected value τ . □

Lemma 43. (*Bounded rejection sampling*) *For $\text{RejectionSample}(m)$ with fallback distribution ν , the output distribution is $(1 - \delta)\pi + \delta\nu$ where $\delta \leq e^{-m/\tau}$.*

Proof. The probability that no sample is accepted after m trials is:

$$\delta = \left(1 - \frac{1}{\tau}\right)^m \leq e^{-m/\tau}$$

where we used the inequality $(1 - x) \leq e^{-x}$ for $x \geq 0$.

With probability $1 - \delta$, a sample is accepted and comes from π (by the first lemma). With probability δ , we fallback to sampling from ν . Therefore, the output distribution is $(1 - \delta)\pi + \delta\nu$. □

C.6. Using rejection sampling for Q^k -means++

In the context of our algorithm, we identify the proposal distribution as $\kappa = \kappa(\cdot|S)$, the target distribution as $\pi = \pi(\cdot|S)$ and the fallback distribution as $\nu = \text{uniform}(\mathcal{X})$. We now show that κ τ -oversamples π for a suitable τ .

Lemma 44. (*Oversampling factor*) *The distribution $\kappa(\cdot|s_1)$ τ -oversamples the distribution $\pi^{\mathcal{L}}(\cdot|S)$ for*

$$\tau = 2\varrho^{-1} \frac{\|\mathcal{X}\|_{\mathbb{F}}^2 + n\|s_1\|^2}{\text{cost}(\mathcal{X}, S)}$$

Proof. The key to this observation is the following use of Cauchy-Schwartz inequality :

$$\text{cost}^{\mathcal{L}}(x, S) \leq \varrho^{-1} \text{cost}(x, S) \leq \varrho^{-1} \|x - s_1\|^2 \leq 2\varrho^{-1} (\|x\|^2 + \|s_1\|^2)$$

Dividing both sides by $\text{cost}^L(\mathcal{X}, S)$, we get

$$\frac{\text{cost}^{\mathcal{L}}(x, S)}{\text{cost}^{\mathcal{L}}(\mathcal{X}, S)} \leq \frac{2\varrho^{-1}(\|x\|^2 + \|s_1\|^2)}{\text{cost}^{\mathcal{L}}(\mathcal{X}, S)} \leq \frac{2\varrho^{-1}(\|x\|^2 + \|s_1\|^2)}{\text{cost}(\mathcal{X}, S)}$$

from which we can see

$$\frac{\text{cost}^{\mathcal{L}}(x, S)}{\text{cost}^{\mathcal{L}}(\mathcal{X}, S)} \leq \left(2\varrho^{-1} \frac{\|\mathcal{X}\|_{\text{F}}^2 + n\|s_1\|^2}{\text{cost}(\mathcal{X}, S)} \right) \frac{\|x\|^2 + \|s_1\|^2}{\|\mathcal{X}\|_{\text{F}}^2 + n\|s_1\|^2}$$

This completes the proof. $\square$

Finally, let us relate this oversampling factor to the quantity $\beta_k(\mathcal{X}) := \frac{\text{opt}_1(\mathcal{X})}{\text{opt}_k(\mathcal{X})}$

Lemma 45. *For the oversampling factor τ in Lemma 44, the following hold:*

- (i) $\mathbf{E}[\tau] \leq 4\varrho^{-1}\beta_k(\mathcal{X})$
- (ii) $\Pr[\tau > 10\varrho^{-1}\beta_k(\mathcal{X})] \leq \frac{1}{4}$

Proof. Since $s_1 \sim \text{UNIFORM}(\mathcal{X})$, we have $\mathbf{E}[\|s_1\|^2] = \frac{1}{n}\|\mathcal{X}\|_{\text{F}}^2$. From Lemma 44, the oversampling factor satisfies:

$$\tau \leq \frac{2\varrho^{-1}(\|\mathcal{X}\|_{\text{F}}^2 + n\|s_1\|^2)}{\text{opt}_k(\mathcal{X})}$$

Proof of (i). Taking expectations:

$$\begin{aligned} \mathbf{E}[\tau] &\leq 2\varrho^{-1} \frac{\|\mathcal{X}\|_{\text{F}}^2 + n\mathbf{E}[\|s_1\|^2]}{\text{opt}_k(\mathcal{X})} \\ &= 2\varrho^{-1} \frac{\|\mathcal{X}\|_{\text{F}}^2 + n \cdot \frac{1}{n} \sum_{x \in \mathcal{X}} \|x\|^2}{\text{opt}_k(\mathcal{X})} \\ &= \frac{4\varrho^{-1}\|\mathcal{X}\|_{\text{F}}^2}{\text{opt}_k(\mathcal{X})} = \frac{4\varrho^{-1}\text{opt}_1(\mathcal{X})}{\text{opt}_k(\mathcal{X})} = 4\varrho^{-1}\beta_k(\mathcal{X}) \end{aligned}$$

where the last equality uses the fact that $\mathcal{X}$ is translated so that its centroid is at the origin, giving $\|\mathcal{X}\|_{\text{F}}^2 = \text{opt}_1(\mathcal{X})$.

Proof of (ii). Define the event $\mathcal{E} := \{\|s_1\|^2 \leq \frac{4}{n}\|\mathcal{X}\|_{\text{F}}^2\}$. By Markov's inequality:

$$\Pr\left[\|s_1\|^2 > \frac{4}{n}\|\mathcal{X}\|_{\text{F}}^2\right] \leq \frac{\mathbf{E}[\|s_1\|^2]}{\frac{4}{n}\|\mathcal{X}\|_{\text{F}}^2} = \frac{\frac{1}{n}\|\mathcal{X}\|_{\text{F}}^2}{\frac{4}{n}\|\mathcal{X}\|_{\text{F}}^2} = \frac{1}{4}$$

so $\Pr[\mathcal{E}] \geq 3/4$. Conditioned on $\mathcal{E}$:

$$\tau \leq \frac{2\varrho^{-1}(\|\mathcal{X}\|_{\text{F}}^2 + n \cdot \frac{4}{n}\|\mathcal{X}\|_{\text{F}}^2)}{\text{opt}_k(\mathcal{X})} = \frac{10\varrho^{-1}\|\mathcal{X}\|_{\text{F}}^2}{\text{opt}_k(\mathcal{X})} = 10\varrho^{-1}\beta_k(\mathcal{X})$$

Therefore $\Pr[\tau > 10\varrho^{-1}\beta_k(\mathcal{X})] \leq \Pr[\mathcal{E}^c] \leq \frac{1}{4}$. $\square$

C.7. Proof of main theorem

We finally need to combine all the results from the previous sections to prove Theorem 6, which claims existence of a fast rejection-sampling based k -means++ while giving an approximation bound of the clustering from the optimal one. Corollary 8 will then follow as a corollary, together with Assumption 1 and Theorem 5.

Let us see how QKMEANS($\mathcal{X}, k, m$) (Algorithm 3) satisfies the claims made by Theorem 6. As seen in Subsection C.1, the preprocessing step takes a time of $O(nD)$. Let us now see how it helps in fast sampling from the distribution $\kappa(\cdot|C)$ which is step 4 of SAMPLE($\mathcal{L}, C, m$).

Here is how this step can be performed:

Procedure 7 SampleDistribution

Input: A set of centers $C \subseteq \mathcal{X}$
Output: A sample according to the distribution $\kappa(\cdot|C)$ defined as $\kappa(x|C) = \frac{\|x\|^2 + \|c_1\|^2}{\|\mathcal{X}\|_F^2 + n\|c_1\|^2}$

- 1: Generate $r \sim \text{UNIFORM}([0, 1])$
- 2: **if** $r \leq \frac{\|\mathcal{X}\|_F^2}{\|\mathcal{X}\|_F^2 + n\|c_1\|^2}$ **then**
- 3: Generate a sample x with probability $\kappa(x)$
- 4: **else**
- 5: Generate a sample $x \sim \text{UNIFORM}(\mathcal{X})$
- 6: **end if**
- 7: **output** x

Lemma 46. *Procedure 7 produces a sample from $\mathcal{X}$ according to the distribution $\kappa(\cdot|C)$. Moreover, it takes $O(\log n)$ time after a one time preprocessing time of $O(nD)$.*

Proof. The probability of a sampled point is as follows:

$$\begin{aligned}
 \Pr[x] &= \Pr \left[r \leq \frac{\|\mathcal{X}\|_F^2}{\|\mathcal{X}\|_F^2 + n\|c_1\|^2} \right] \Pr[x \sim \kappa(\cdot)] \\
 &\quad + \Pr \left[r > \frac{\|\mathcal{X}\|_F^2}{\|\mathcal{X}\|_F^2 + n\|c_1\|^2} \right] \Pr[x \sim \text{UNIFORM}(\mathcal{X})] \\
 &= \frac{\|\mathcal{X}\|_F^2}{\|\mathcal{X}\|_F^2 + n\|c_1\|^2} \frac{\|x\|^2}{\|\mathcal{X}\|_F^2} + \frac{n\|c_1\|^2}{\|\mathcal{X}\|_F^2 + n\|c_1\|^2} \frac{1}{n} \\
 &= \frac{(\|x\|^2 + \|c_1\|^2)}{(\|\mathcal{X}\|_F^2 + n\|c_1\|^2)} = \kappa(x|C)
 \end{aligned}$$

The time complexity follows from Corollary 25. □

Next, let us recall from Lemma 26 that the LSH-based ANNS data structure supports (conditioned on success with probability $1 - 1/t$) a $\text{Insert}(\mathcal{L}, p)$ operation in time $O(D \log \eta(t \log \eta)^e)$ and a $\text{Query}(\mathcal{L}, p)$ operation in time $O(D \log \eta(t \log \eta)^e)$, where t is the size of the set of points P to be inserted. In our case, the $\text{QKMEANS}(\mathcal{X}, k, m)$ algorithm only inserts the chosen cluster centers in the dataset, i.e. $t = k$ (Note that the $\text{Query}(\mathcal{L}, p)$ operation can be made for any $p \in \mathcal{X}$).

Also, recall that here, D is the dimension of the dataset after applying the linear transformation from (Makarychev et al., 2019), so that $D = O(\log k)$, with a constant distortion in the clustering cost w.r.t. any centers.

Now, observe that the INSERT operation is invoked k times, and the SAMPLE subroutine is invoked k times from the QKMEANS algorithm. Within the SAMPLE subroutine, a QUERY operation and a sampling from $\kappa(\cdot|C)$ operation are invoked at most $m \ln k$ times. This means that there are a total of k INSERT operations, $mk \ln k$ QUERY operations and $mk \ln k$ sampling operations from $\kappa(\cdot|C)$.

Thus we obtain a time complexity (after the processing step) of

$$O(kD \log \eta(t \log \eta)^e) + O(mk(\ln k)D \log \eta(t \log \eta)^e) + O(mk \ln k \log n)$$

which after substituting values of D, t can be simplified to $O(m(\log^2 k)(k \log \eta)^{1+e} + mk \ln k \log n)$. This completes the time complexity analysis.

Now, let us analyse the approximation guarantee. From Theorem 38, the (ϱ, δ) - k -means++ follows an approximation guarantee of:

$$\mathbf{E}[\text{cost}(\mathcal{X}, C)] \in O(\varrho^{-2} \log k) \text{opt}_k(\mathcal{X}) + \delta \cdot O(k + \varrho^{-2} \log k) \text{opt}_1(\mathcal{X})$$

where δ is the probability of choosing a center via the uniform distribution (which happens due to the failure of rejection sampling). It remains to bound δ . From Lemma 43 and Lemma 44, we have $\delta \leq e^{-m \ln k / \tau}$ where $\tau = 2\varrho^{-1} \beta(\mathcal{X}, S)$. By part (ii) of Lemma 45, we have $\tau \leq 10\varrho^{-1} \beta_k(\mathcal{X})$ with probability at least $3/4$, where this bound is independent of the centers S chosen by the algorithm. Let $c = \lceil \log(\text{opt}_1(\mathcal{X}) / \text{opt}_k(\mathcal{X})) \rceil = \lceil \log \beta_k(\mathcal{X}) \rceil$. We repeat the entire seeding procedure c independent times and return the best clustering. Since $\Pr[\tau \leq 10\varrho^{-1} \beta_k(\mathcal{X})] \geq 3/4$ independently across runs, at least one run satisfies the bound with probability at least $1 - (\frac{1}{4})^c$. Recalling that we always have $\mathbf{E}[\text{cost}(\mathcal{X}, S)] \leq 2\text{opt}_1(\mathcal{X})$

since the first center is chosen uniformly at random, we can write the final expected cost, after taking the best clustering over c repetitions as $\mathbf{E}[\text{cost}(\mathcal{X}, S)] \leq (1 - 4^{-c})\mathbf{E}[\text{cost}(\mathcal{X}, C) | \tau \leq 10\varrho^{-1}\beta_k(\mathcal{X})] + 4^{-c} \cdot 2\text{opt}_1(\mathcal{X})$. By our choice of c , this only adds a factor of $O(1)\text{opt}_k(\mathcal{X})$ to the approximation guarantee.

Thus the total expected cost satisfies:

$$\mathbf{E}[\text{cost}(\mathcal{X}, C)] \in \underbrace{O(\varrho^{-2}) \log k \cdot \text{opt}_k(\mathcal{X})}_{k\text{-means++ guarantee}} + \underbrace{O(\varrho^{-2}) \cdot k^{-\Omega(\varrho m / \beta_k(\mathcal{X}))} \cdot \text{opt}_1(\mathcal{X})}_{\text{exponentially decaying additive term}}$$

which completes the proof of Theorem 6.

Corollary 7 follows as a corollary from Theorem 6; if we allow $m = \infty$ in $\text{SAMPLE}(\mathcal{L}, C, m)$, we can see that the ‘break’ statement will be issued in expected time $O(\tau)$, since we need $\tau = 2\varrho^{-1}\beta(\mathcal{X}, C)$, $\mathbf{E}[\tau] \leq 4\varrho^{-1}\beta_k(\mathcal{X})$ (from Lemma 45) samples of distribution $\kappa(\cdot | C)$ in expectation to get a desired sample by Lemma 42.

Finally, Corollary 8 can be obtained as a corollary of Corollary 7 assuming the manifold hypothesis. Theorem 5 gives us $\beta_k(\mathcal{X}) \leq \beta_0 k^\varepsilon, \eta(\mathcal{X}) \leq \eta_0 n^{3\varepsilon/2}$ for quantization exponent ε . Substituting these in the time complexity bound of Corollary 7 gives Corollary 8, keeping in mind that we always have $\varepsilon \in O(1)$.

D. Empirical Study

In this section, we empirically evaluate the extent to which the theoretical predictions developed in this paper are reflected in real-world data.

Compute environment. All experiments were conducted on a dual-socket machine equipped with IntelXeonGold5220R CPUs (48 physical cores total, 2.20 GHz) and 96 hardware threads, running Linux on x86-64 architecture. Unless otherwise stated, all experiments were executed in a CPU-only setting with a fixed number of threads. Implementations were compiled with aggressive compiler optimizations, and all reported runtimes are wall-clock times averaged over multiple random seeds.

Datasets. We use datasets across several domains for our empirical evaluation. We include popular image datasets, both as raw pixels as well as image embeddings which are produced through joint vision-language models such as CLIP-ViT-B/32; text datasets in the form of text embeddings generated by the All-MiniLM-L6-v2 model; tabular datasets which record sensor data as well as data generated by scientific experiments. The details of the datasets used are presented in Table 2.

CATEGORY	DATASET	n	d
RAW IMAGES	MNIST	60,000	784
	FMNIST	60,000	784
	CIFAR10	50,000	3,072
	CIFAR100	50,000	3,072
CLIP ViT-B/32 IMAGE EMBEDDINGS	MNIST	60,000	512
	FMNIST	60,000	512
	CIFAR10	50,000	512
	CIFAR100	50,000	512
ALL-MINILM-L6-v2 TEXT EMBEDDINGS	REDDIT	100,000	384
	STACKEXCHANGE	20,000	384
TABULAR	SUSY	5,000,000	18
	HAR	10,299	561

D.1. Scaling Laws

We test the hypothesis that the data-dependent parameters $\beta_k = \text{opt}_1 / \text{opt}_k$ and η_k grow polynomially with the number of clusters k , as predicted by optimal quantization theory. For each dataset, we compute k -means clusterings for $k \in \{5, 10, 50, 100, 250, 500, 750, 1000\}$ using 10 independent runs. We fit linear regressions to $\log \beta_k$ and $\log \eta_k$ versus $\log k$ and report slopes, R^2 , and 95% confidence intervals. Across datasets and modalities, both quantities exhibit reasonable linear behavior in log-log coordinates. The plots are shown in Figures 7 and 8. Table 4 provides the estimated quantization exponents and R^2 scores for the best fit estimates. The high R^2 scores ($\gtrsim 0.95$) show that the scaling laws fit well on the data.

D.2. Intrinsic Dimension

We test the hypothesis that the empirically estimated quantization exponent $\hat{\varepsilon}$ is inversely proportional to the intrinsic dimension. Independent intrinsic dimension estimates $\hat{d}_{\text{MLE}}$ are obtained by using the popular maximum likelihood method (MLE) (Levina & Bickel, 2005). For a dataset $\mathcal{X} = \{x_1, \dots, x_n\}$, let $T_j(x_i)$ denote the distance of the j th nearest neighbor of x_i from x_i . For a hyperparameter k , the ID estimate is calculated as :

$$\hat{d}_{\text{MLE}}(\mathcal{X}, k) := \frac{1}{n} \sum_{i=1}^n \left(\frac{1}{k-1} \sum_{j=1}^{k-1} \log \frac{T_k(x_i)}{T_j(x_i)} \right)^{-1}$$

For each dataset we take 10 independent samples each of size $n' = 10,000$ to estimate the ID for a particular value of k . This is repeated for $k \in \{5, 10, 20, 50, 100\}$ and the average is reported. We plot the estimates $\hat{\varepsilon}$ of the quantization exponent against $\hat{d}_{\text{MLE}}$ in Figure 5, and the estimated dimension $\hat{d}_{\varepsilon} = 2/\hat{\varepsilon}$ against $\hat{d}_{\text{MLE}}$. We observe significant agreement with Pearson’s $R = 0.92$ and an R^2 score of 0.84. It is to be noted that ID estimation in itself is a challenging problem with several proposed estimators in the literature. We refer the reader to the surveys (Camastra & Staiano, 2016; Binnie et al.,

Table 4. Scaling law parameters across datasets. ε is estimated from $\beta_k \sim k^\varepsilon$. R_β^2 and R_η^2 are the coefficients of determination for the β and η scaling fits respectively.

Dataset	$\hat{\varepsilon}$	$\hat{R}_\beta^2$	$\hat{R}_\eta^2$
MNIST	0.1449	0.9998	0.9944
FASHION-MNIST	0.1808	0.9847	0.9702
CIFAR-10	0.0862	0.9945	0.9661
CIFAR-100	0.0961	0.9919	0.9659
MNIST-CLIP	0.1759	0.9980	0.9981
FMNIST-CLIP	0.1665	0.9887	0.9809
CIFAR-10-CLIP	0.1101	0.9898	0.9737
CIFAR-100-CLIP	0.1166	0.9988	0.9938
REDDIT	0.0622	0.9899	0.9814
STACKEXCHANGE	0.0565	0.9826	0.9898
HAR	0.1894	0.9984	0.8791
SUSY	0.2644	0.9993	0.9693

Table 5. Comparison of intrinsic dimension estimates. ε is the scaling exponent from $\beta \sim k^\varepsilon$. $d_\varepsilon = 2/\varepsilon$ is the intrinsic dimension estimate from the scaling law. d_{MLE} is the maximum likelihood estimate of intrinsic dimension from (Levina & Bickel, 2005).

Dataset	ε	$\hat{d}_\varepsilon$	$\hat{d}_{MLE}$
MNIST	0.1449	13.80	14.25
FASHION-MNIST	0.1808	11.06	15.64
CIFAR-10	0.0862	23.21	28.39
CIFAR-100	0.0961	20.81	25.69
MNIST-CLIP	0.1759	11.37	14.43
FMNIST-CLIP	0.1665	12.01	16.22
CIFAR-10-CLIP	0.1101	18.16	22.05
CIFAR-100-CLIP	0.1166	17.15	20.13
REDDIT	0.0622	32.13	29.22
HAR	0.1894	10.56	14.75
SUSY	0.2644	7.56	7.88
STACKEXCHANGE	0.0565	35.40	28.48

2025) for further discussion on ID estimators. We have chosen the MLE estimator of (Levina & Bickel, 2005) since it is one of the most established and widely studied estimators in the literature.

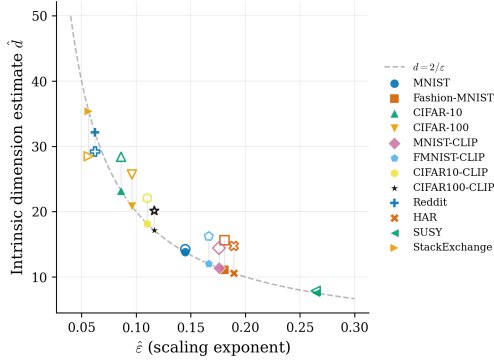


Figure 5. Relationship between ε (from β scaling) and d_{MLE} .

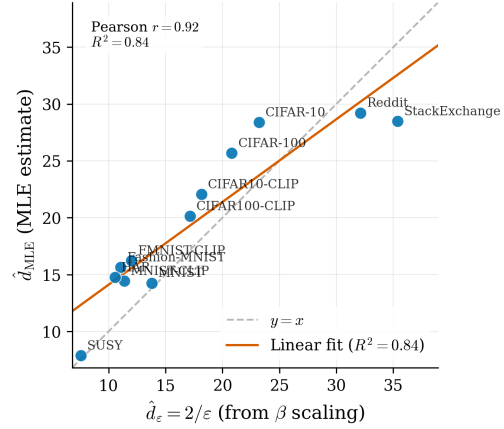


Figure 6. Comparison of d_ε and d_{MLE} .

D.3. Effect of manifold structure

To assess the robustness of the quantization scaling law $\beta_k \sim k^\varepsilon$ under distributional perturbation, we inject isotropic Gaussian noise at 20 linearly-spaced noise-to-signal ratios (NSR) from 0 to 2 across four datasets (MNIST, MNIST-CLIP, HAR, STACKEXCHANGE), where $\mathcal{X}_{\text{noisy}} = X + \text{NSR} \cdot \sigma(\mathcal{X}) \cdot \mathcal{N}(0, 1)$, and re-fit the power-law exponent ε at each level. Across all datasets, ε decays monotonically and convexly toward $2/D$, with the majority of degradation occurring at low noise levels ($\text{NSR} < 0.5$), after which the exponent plateaus well above the ambient-dimension limit $2/D$. The rate of decay varies with intrinsic structure: compact, low-dimensional representations such as MNIST-CLIP ($d \approx 11$, $D = 512$) lose most of their scaling signal by $\text{NSR} \approx 0.5$, while higher-dimensional datasets like StackExchange ($d \approx 39$, $D = 384$) degrade more gradually. This confirms that the scaling exponent captures genuine geometric structure rather than statistical artifacts, and suggests that ε could serve as a noise diagnostic — its sensitivity to perturbation is itself informative about the signal-to-noise regime of the data. The plots are shown in Figure 10.

D.4. Rejection sampling analysis

Rejection Rate Experiment. We investigate the rejection sampling behavior of QKMEANS across 12 datasets by varying the chain length $m \in \{1, 2, 3, 5, 7, 10, 15, 20, 30, 50\}$ and number of clusters $k \in \{10, 50, 100\}$, averaging over 5 independent

runs. For each center selection step, QKMEANS draws up to $m \ln(k+1)$ proposals from the distribution $\kappa(x) \propto \|x\|^2 + \|c_1\|^2$ and accepts a proposal with probability proportional to its distance to the nearest existing center; if all proposals are rejected, the algorithm falls back to uniform random selection. Low-dimensional and well-separated datasets (MNIST, FashionMNIST, SUSY) exhibit low rejection even at $m = 1$ and reach zero rejection by $m \approx 5-10$. High-dimensional raw-pixel datasets (CIFAR-10, CIFAR-100) show moderate rejection at small m ($\sim 20-30$) but converge to zero by $m \approx 60-80$. CLIP embeddings present the most challenging setting: FMNIST-CLIP and MNIST-CLIP maintain elevated rejection rates ($\sim 75-90$ at $m = 1$), with MNIST-CLIP retaining $\sim 10-15$ rejection even at $m = 50$, reflecting the tighter concentration of norm-based proposal distributions in these embedding spaces. In contrast, text embedding datasets (StackExchange, Reddit) exhibit near-zero rejection across all m values, indicating that their κ distribution closely approximates the D^2 distribution. Across all datasets, larger k generally yields higher rejection fractions at a given m , as later center selections must distinguish among increasingly fine-grained cluster structure. These results suggest that $m \approx 10-20$ suffices for most practical settings, while highly concentrated embedding datasets may benefit from larger chain lengths. The plots are shown in Figure 11.

D.5. Effect of ANNS used

Although our theoretical analysis uses Locality Sensitive Hashing for implementing nearest neighbor search, there are several other methods for doing the same. Our algorithm uses the ANNS algorithm as a black-box and hence, it can be easily replaced by a different method. We test several ANNS methods on the datasets to study the runtime-solution quality tradeoffs. We include brute-force search, hierarchically navigable small worlds (Malkov & Yashunin, 2020), locality sensitive hasing (Indyk & Motwani, 1998), inverted file index (Jégou et al., 2011b) and random projection trees (Dasgupta & Freund, 2008). For small $k \approx 50$, Qkmeans is fastest with brute-force nearest-neighbor search, which outperforms LSH and tree-based ANNS by $2-4\times$ even in very high dimensions, since ANNS overhead dominates at low k . As k increases, this trend reverses: tree-based methods (e.g., RP-trees, HNSW) become increasingly advantageous by amortizing query costs across many seeding steps. The results are in Table 6.

D.6. Comparison with Prior Art

In our comparative analysis, we consider the following algorithms :

- AFKMC² from (Bachem et al., 2016a)
- RejectionSampling from (Cohen-Addad et al., 2020a)
- PRONECoreset from (Charikar et al., 2023)
- FastKMeans from (Draganov et al., 2024)

We followed the reference implementations from <https://github.com/adriangoef/afkmc2> for AFKMC², from <https://git.informatik.uni-hamburg.de/0moin/google-research/-/tree/masterfor> RejectionSampling, from <https://github.com/boredoms/prone> for PRONECoreset and from <https://github.com/Andrew-Draganov/Fast-Coreset-Generation> for FastKMeans. We use the default hyperparameters which are specified in these implementations. For implementing Qkmeans, we use the HNSW ANNS algorithm.

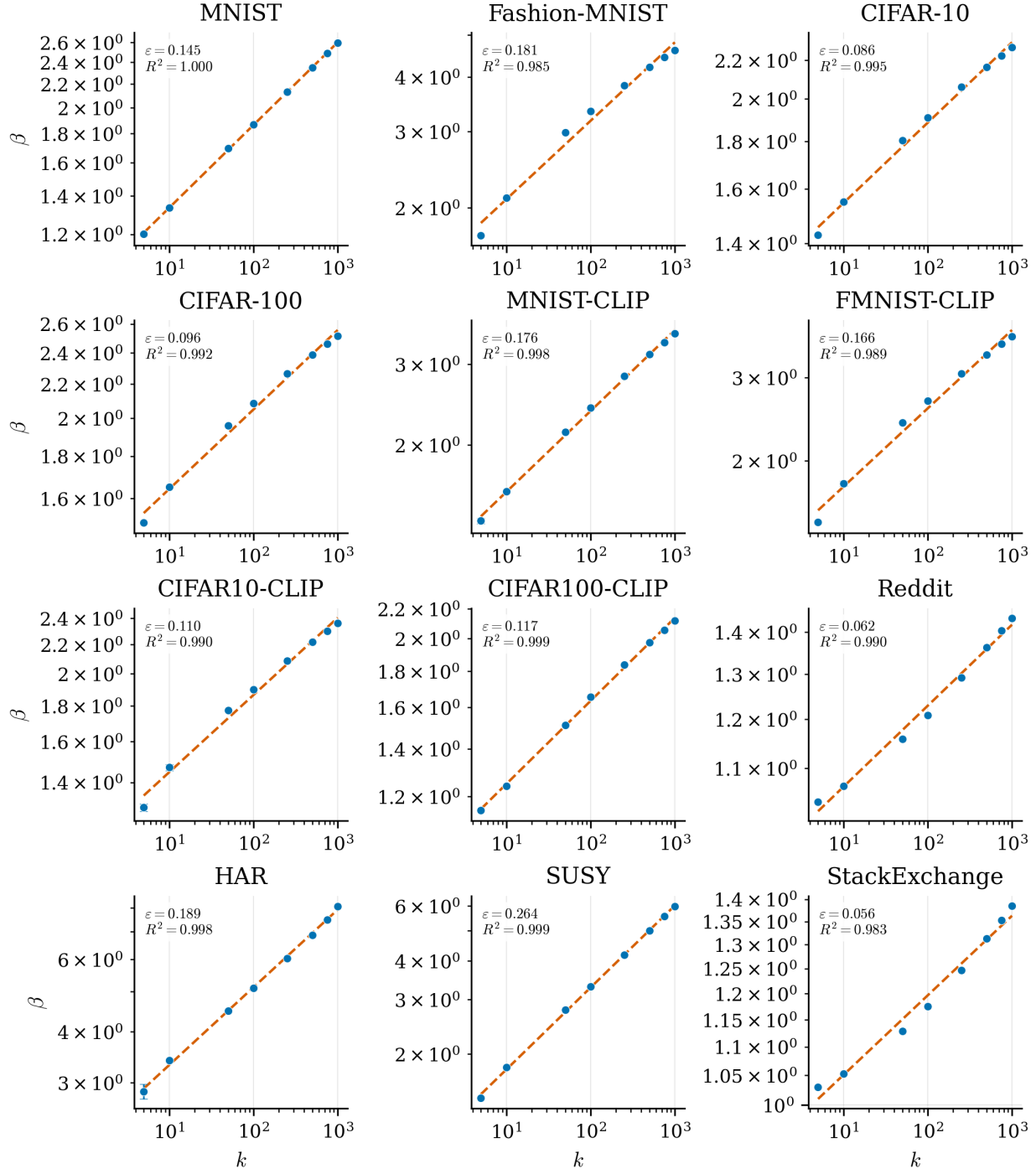
Figure 9 shows the seeding time and seeding quality tradeoffs for various algorithms across datasets. These results are also tabulated in Tables 7-18.

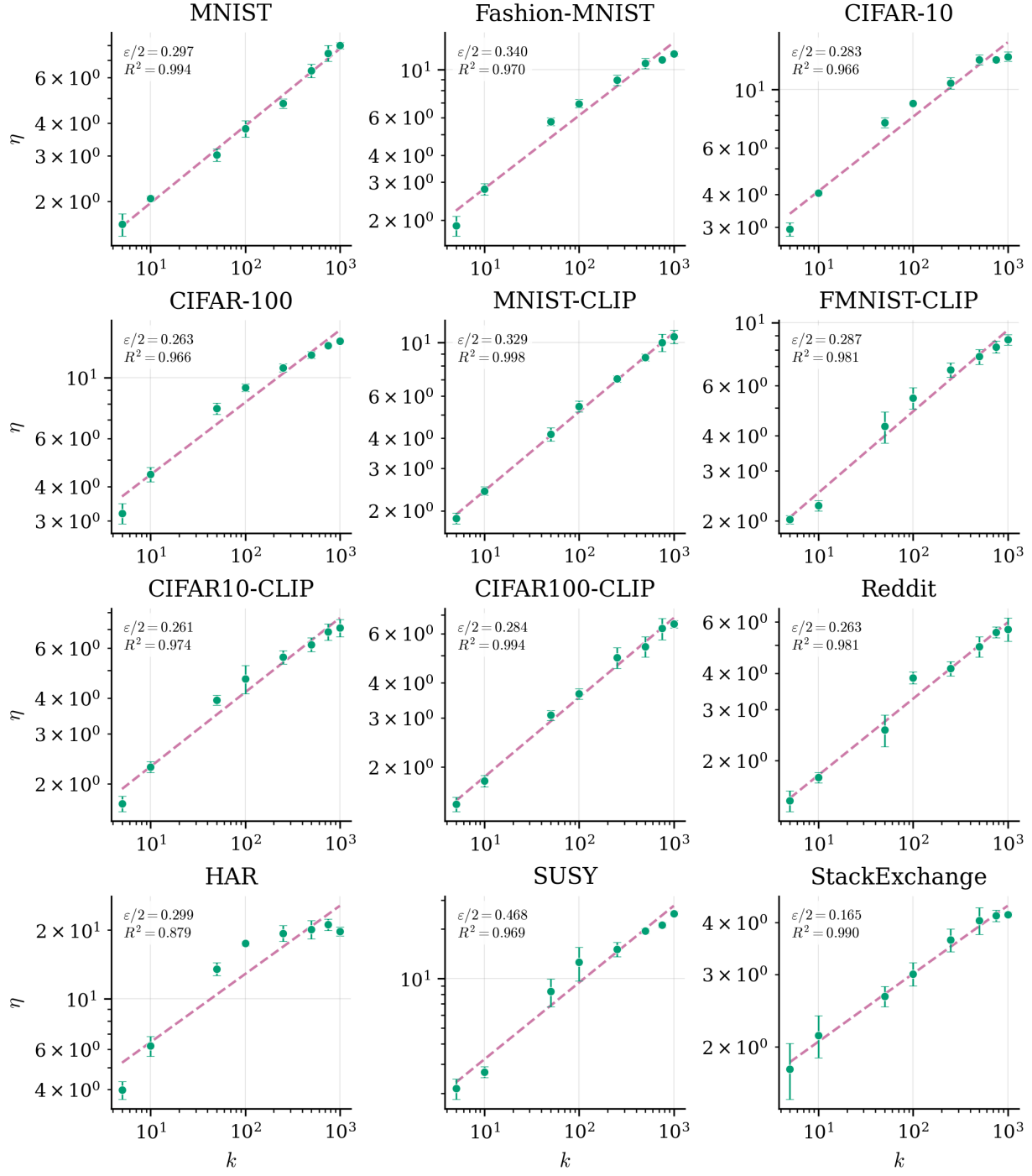
D.7. Summary of Empirical Findings

Across all datasets, we observe consistent evidence for quantization-theoretic scaling, a strong inverse relationship between intrinsic dimension and scaling exponents, and stable rejection-sampling behavior. These results collectively support the central premise that geometric structure governs the empirical behavior of fast k -means seeding algorithms.

Table 6. QKMEANS seeding time (ms) with different ANNS methods across all datasets and k values.

DATASET	k	HNSW	BRUTEFORCE	LSH	IVF	RPTREE
MNIST ($d = 784$)	50	12.6	9.4	16.8	10.5	12.7
	200	43.3	24.8	46.6	22.1	23.9
	1000	526.6	544.0	693.1	348.8	93.6
FASHION-MNIST ($d = 784$)	50	14.8	8.8	19.4	9.1	12.5
	200	73.1	48.4	99.0	46.2	43.3
	1000	852.6	1506.8	1961.1	1256.3	233.6
CIFAR-10 ($d = 3072$)	50	36.1	31.4	113.1	40.6	60.6
	200	341.5	301.6	681.5	327.4	231.3
	1000	5242.8	8899.1	10951.5	8596.0	1396.1
CIFAR-100 ($d = 3072$)	50	31.8	29.3	105.3	33.4	52.3
	200	264.6	241.6	556.7	253.8	189.6
	1000	4187.5	7065.1	8738.9	7012.6	1156.1
MNIST-CLIP ($d = 512$)	50	33.0	15.6	78.4	23.8	34.5
	200	405.5	193.5	878.2	233.9	207.6
	1000	4702.3	6664.2	26362.4	6985.7	1052.7
FMNIST-CLIP ($d = 512$)	50	21.9	10.6	28.4	11.9	16.8
	200	155.7	76.6	299.7	123.7	72.2
	1000	1828.8	2510.2	4885.6	2689.8	387.9
CIFAR10-CLIP ($d = 512$)	50	11.8	5.5	16.2	11.8	13.2
	200	73.0	37.9	65.5	38.9	27.9
	1000	735.5	825.2	960.7	856.4	141.6
CIFAR100-CLIP ($d = 512$)	50	13.9	7.2	13.0	7.2	10.4
	200	59.6	31.3	52.7	30.4	24.8
	1000	695.4	707.8	816.7	742.9	120.4
REDDIT ($d = 384$)	50	9.5	4.1	6.4	5.0	6.1
	200	21.2	12.2	14.4	9.7	9.7
	1000	221.7	93.9	40.8	96.1	26.2
STACKEXCHANGE ($d = 384$)	50	7.6	3.9	7.5	5.9	5.0
	200	24.0	10.2	13.9	12.6	9.5
	1000	225.4	84.8	39.1	87.6	24.4
HAR ($d = 561$)	50	19.6	8.6	37.0	13.5	20.2
	200	171.8	107.8	212.9	113.1	87.5
	1000	2186.7	3958.6	4748.6	3947.5	556.0
SUSY ($d = 19$)	50	6.5	2.5	3.4	1.5	3.0
	200	35.8	6.1	12.1	6.7	5.3
	1000	370.5	68.3	85.7	81.6	40.0


 Figure 7. Scaling Laws for β_k vs k


 Figure 8. Scaling Laws for η_k vs k

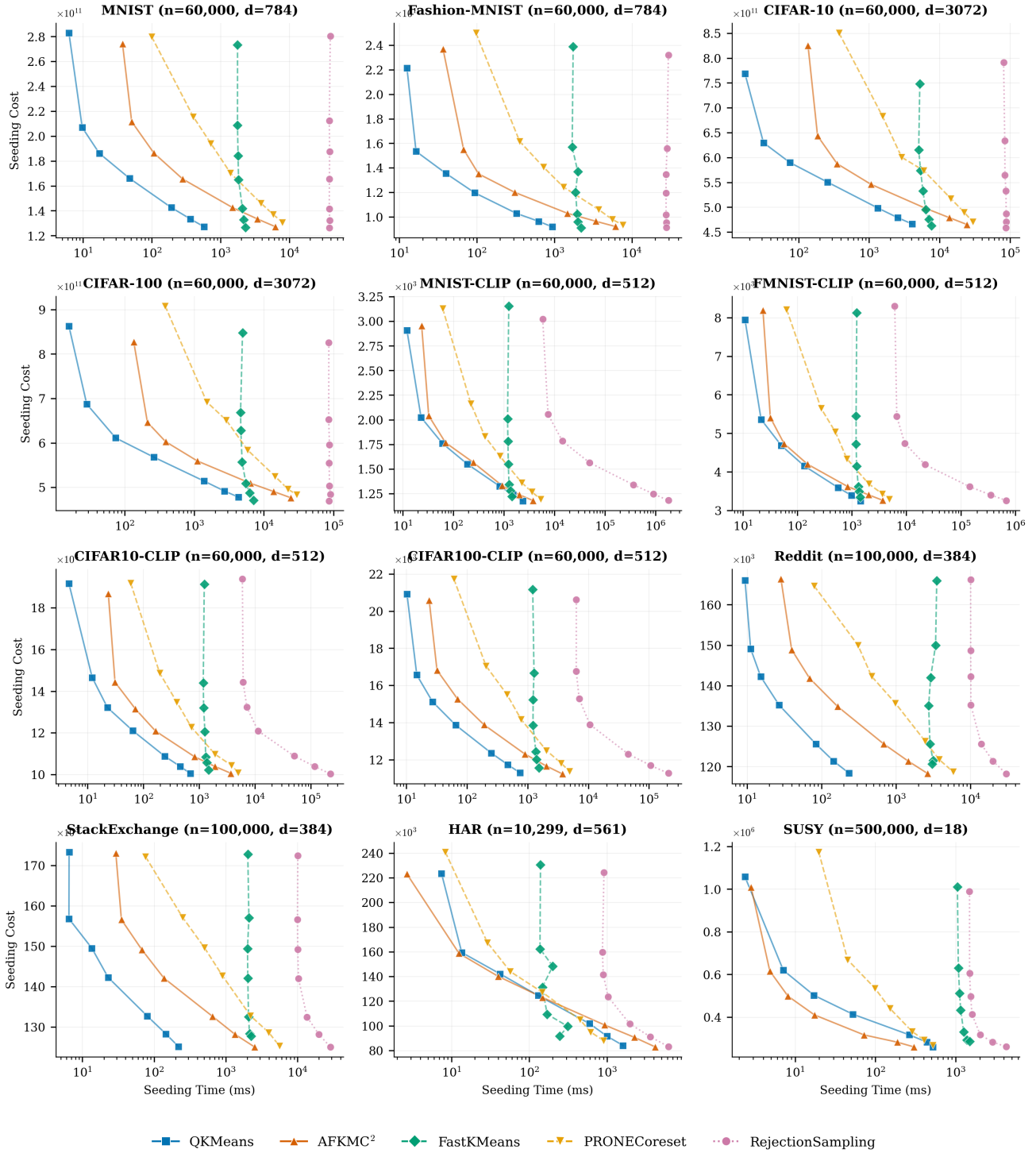


Figure 9. Plots for seeding time and seeding quality tradeoffs for various algorithms

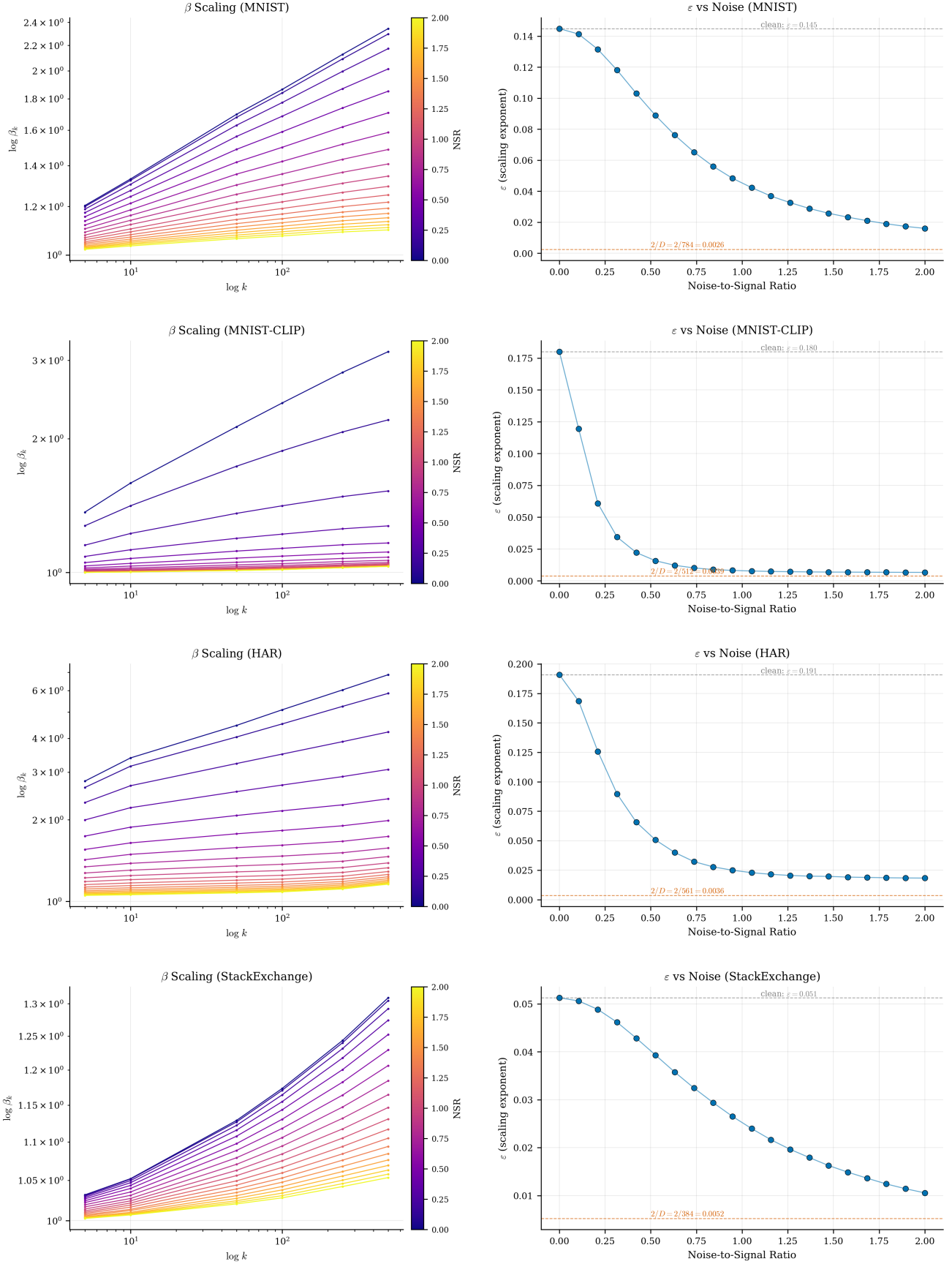


Figure 10. Effect of noise on the scaling laws, demonstrated on 4 datasets

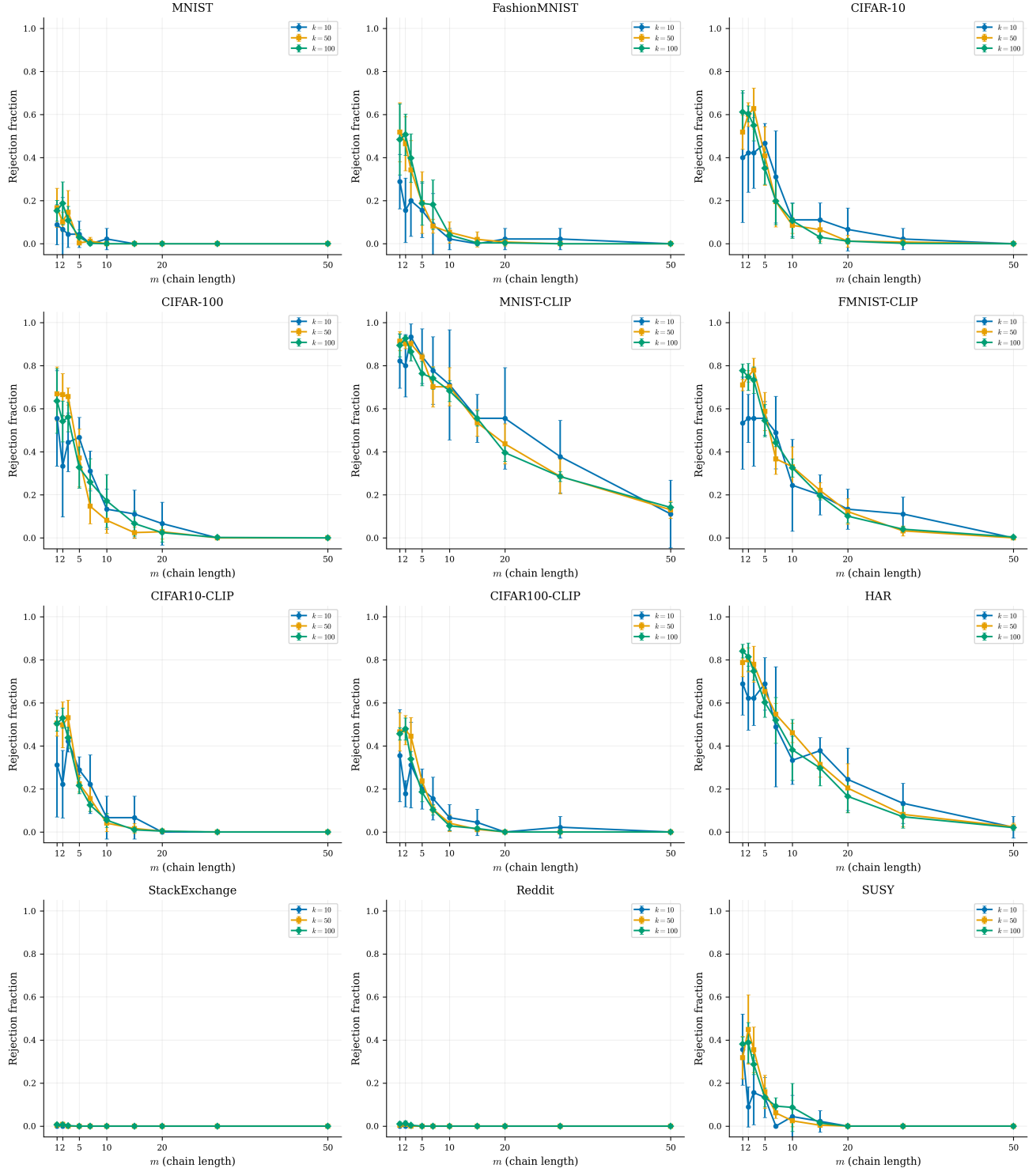

 Figure 11. Fraction of samples rejected vs the chain length m

Table 7. Seeding benchmark on mnist. Time in ms, cost $\times 10^{11}$.

Algorithm	$k = 10$		$k = 50$		$k = 100$		$k = 200$		$k = 500$		$k = 750$		$k = 1000$	
	Time	Cost	Time	Cost	Time	Cost	Time	Cost	Time	Cost	Time	Cost	Time	Cost
AFKMC ²	38	2.74	51	2.12	107	1.86	281	1.65	1.5s	1.43	3.4s	1.33	6.2s	1.27
PRONECoreset	99	2.80	400	2.16	719	1.94	1.4s	1.70	3.9s	1.46	5.8s	1.37	7.9s	1.31
FastKMeans	1.8s	2.73	1.8s	2.09	1.8s	1.84	1.8s	1.65	2.1s	1.42	2.2s	1.33	2.3s	1.27
RejectionSampling	39.2s	2.80	37.8s	2.12	38.3s	1.88	38.0s	1.65	37.8s	1.41	38.3s	1.32	38.0s	1.26
Qkmeans	6.2	2.83	9.7	2.07	17	1.86	48	1.66	192	1.43	365	1.33	573	1.27

Table 8. Seeding benchmark on fmnist. Time in ms, cost $\times 10^{11}$.

Algorithm	$k = 10$		$k = 50$		$k = 100$		$k = 200$		$k = 500$		$k = 750$		$k = 1000$	
	Time	Cost	Time	Cost	Time	Cost	Time	Cost	Time	Cost	Time	Cost	Time	Cost
AFKMC ²	37	2.37	67	1.55	105	1.35	308	1.20	1.5s	1.03	3.4s	0.96	6.1s	0.92
PRONECoreset	97	2.50	356	1.62	722	1.41	1.3s	1.24	3.7s	1.06	5.6s	0.98	7.7s	0.93
FastKMeans	1.7s	2.39	1.7s	1.57	2.0s	1.37	1.9s	1.20	2.0s	1.02	2.0s	0.96	2.2s	0.91
RejectionSampling	29.6s	2.32	28.5s	1.56	27.6s	1.35	27.6s	1.19	27.7s	1.02	27.9s	0.95	27.9s	0.91
Qkmeans	13	2.21	16	1.53	40	1.35	94	1.20	325	1.03	627	0.96	941	0.92

Table 9. Seeding benchmark on cifar10. Time in ms, cost $\times 10^{11}$.

Algorithm	$k = 10$		$k = 50$		$k = 100$		$k = 200$		$k = 500$		$k = 750$		$k = 1000$	
	Time	Cost	Time	Cost	Time	Cost	Time	Cost	Time	Cost	Time	Cost	Time	Cost
AFKMC ²	136	8.25	186	6.44	352	5.87	1.1s	5.46	6.2s	4.99	13.8s	4.79	24.6s	4.65
PRONECoreset	374	8.50	1.6s	6.83	2.9s	6.00	6.0s	5.74	14.5s	5.17	22.4s	4.90	29.7s	4.70
FastKMeans	5.3s	7.48	5.1s	6.15	5.4s	5.74	5.8s	5.33	6.4s	4.95	7.1s	4.76	7.7s	4.63
RejectionSampling	81.4s	7.91	84.5s	6.33	85.1s	5.65	87.8s	5.32	88.5s	4.87	89.1s	4.71	87.8s	4.58
Qkmeans	17	7.68	32	6.30	75	5.90	258	5.50	1.3s	4.98	2.5s	4.79	4.1s	4.66

Table 10. Seeding benchmark on cifar100. Time in ms, cost $\times 10^{11}$.

Algorithm	$k = 10$		$k = 50$		$k = 100$		$k = 200$		$k = 500$		$k = 750$		$k = 1000$	
	Time	Cost	Time	Cost	Time	Cost	Time	Cost	Time	Cost	Time	Cost	Time	Cost
AFKMC ²	135	8.27	212	6.46	389	6.03	1.1s	5.59	6.4s	5.09	13.8s	4.91	24.4s	4.77
PRONECoreset	382	9.08	1.5s	6.92	2.9s	6.51	5.9s	5.84	14.5s	5.25	22.2s	4.96	29.7s	4.83
FastKMeans	4.9s	8.48	4.6s	6.69	4.7s	6.28	4.9s	5.57	5.5s	5.09	6.3s	4.87	7.2s	4.71
RejectionSampling	85.9s	8.26	85.5s	6.53	87.7s	5.96	86.7s	5.54	87.9s	5.04	90.2s	4.84	86.6s	4.70
Qkmeans	16	8.63	29	6.87	74	6.11	262	5.68	1.4s	5.14	2.7s	4.92	4.3s	4.78

Table 11. Seeding benchmark on mnist_clip. Time in ms, cost.

Algorithm	$k = 10$		$k = 50$		$k = 100$		$k = 200$		$k = 500$		$k = 750$		$k = 1000$	
	Time	Cost	Time	Cost	Time	Cost	Time	Cost	Time	Cost	Time	Cost	Time	Cost
AFKMC ²	24	2955	33	2041	70	1768	248	1566	927	1330	2.0s	1239	3.8s	1178
PRONECoreset	62	3131	224	2162	422	1830	838	1629	2.3s	1361	3.6s	1265	5.4s	1191
FastKMeans	1.3s	3156	1.2s	2009	1.2s	1781	1.2s	1550	1.3s	1340	1.4s	1279	1.5s	1221
RejectionSampling	5.9s	3021	7.5s	2055	14.4s	1783	49.9s	1563	363.8s	1340	912.2s	1246	1800.0s	1180
Qkmeans	12	2907	23	2024	61	1759	190	1550	828	1325	1.5s	1239	2.4s	1175

Table 12. Seeding benchmark on fmnist_clip. Time in ms, cost .

Algorithm	$k = 10$		$k = 50$		$k = 100$		$k = 200$		$k = 500$		$k = 750$		$k = 1000$	
	Time	Cost	Time	Cost	Time	Cost	Time	Cost	Time	Cost	Time	Cost	Time	Cost
AFKMC ²	23	8185	32	5407	57	4732	153	4201	839	3620	2.0s	3407	3.7s	3262
PRONECoreset	63	8210	272	5646	498	5038	812	4339	2.0s	3689	3.6s	3430	4.9s	3294
FastKMeans	1.2s	8128	1.2s	5445	1.2s	4720	1.2s	4143	1.3s	3622	1.4s	3490	1.4s	3348
RejectionSampling	6.1s	8301	6.7s	5437	9.3s	4742	22.1s	4188	145.5s	3616	355.1s	3401	685.6s	3257
Qkmeans	11	7944	22	5361	51	4682	134	4155	555	3596	976	3394	1.4s	3249

Table 13. Seeding benchmark on cifar10_clip. Time in ms, cost $\times 10^2$.

Algorithm	$k = 10$		$k = 50$		$k = 100$		$k = 200$		$k = 500$		$k = 750$		$k = 1000$	
	Time	Cost	Time	Cost	Time	Cost	Time	Cost	Time	Cost	Time	Cost	Time	Cost
AFKMC ²	24	187	31	144	72	131	165	121	823	108	1.9s	104	3.7s	101
PRONECoreset	59	192	195	149	396	135	720	123	1.9s	110	3.8s	104	5.0s	101
FastKMeans	1.2s	191	1.2s	144	1.2s	132	1.3s	120	1.3s	108	1.4s	106	1.5s	102
RejectionSampling	5.9s	194	6.1s	144	7.2s	132	11.4s	121	50.5s	109	117.1s	104	224.0s	100
Qkmeans	4.6	192	12	146	23	132	65	121	240	109	453	104	695	100

Table 14. Seeding benchmark on cifar100_clip. Time in ms, cost $\times 10^2$.

Algorithm	$k = 10$		$k = 50$		$k = 100$		$k = 200$		$k = 500$		$k = 750$		$k = 1000$	
	Time	Cost	Time	Cost	Time	Cost	Time	Cost	Time	Cost	Time	Cost	Time	Cost
AFKMC ²	24	206	32	168	70	153	191	139	906	123	2.0s	117	3.7s	112
PRONECoreset	61	218	206	170	455	155	782	142	2.0s	125	3.6s	118	4.9s	114
FastKMeans	1.2s	212	1.3s	167	1.2s	152	1.2s	138	1.4s	124	1.4s	120	1.5s	116
RejectionSampling	6.3s	206	6.3s	168	7.1s	153	10.6s	139	45.4s	123	106.6s	117	208.3s	113
Qkmeans	10	209	15	166	27	151	65	139	252	124	470	117	745	113

Table 15. Seeding benchmark on reddit. Time in ms, cost $\times 10^2$.

Algorithm	$k = 10$		$k = 50$		$k = 100$		$k = 200$		$k = 500$		$k = 750$		$k = 1000$	
	Time	Cost	Time	Cost	Time	Cost	Time	Cost	Time	Cost	Time	Cost	Time	Cost
AFKMC ²	29	1664	40	1489	69	1418	165	1348	687	1256	1.5s	1214	2.7s	1183
PRONECoreset	80	1647	309	1500	472	1423	982	1357	2.5s	1263	3.8s	1218	5.9s	1187
FastKMeans	3.5s	1659	3.4s	1500	2.9s	1420	2.8s	1350	2.9s	1256	3.2s	1215	3.1s	1207
RejectionSampling	10.1s	1662	10.1s	1486	10.2s	1423	10.2s	1352	14.0s	1255	20.2s	1213	30.5s	1182
Qkmeans	9.4	1660	11	1491	15	1422	27	1352	85	1256	146	1213	235	1183

Table 16. Seeding benchmark on stackexchange. Time in ms, cost $\times 10^2$.

Algorithm	$k = 10$		$k = 50$		$k = 100$		$k = 200$		$k = 500$		$k = 750$		$k = 1000$	
	Time	Cost	Time	Cost	Time	Cost	Time	Cost	Time	Cost	Time	Cost	Time	Cost
AFKMC ²	29	1730	35	1566	68	1492	137	1421	655	1326	1.3s	1282	2.5s	1251
PRONECoreset	76	1722	250	1572	504	1497	900	1427	2.2s	1328	4.0s	1286	5.6s	1253
FastKMeans	2.0s	1728	2.1s	1571	2.0s	1494	2.0s	1421	2.1s	1326	2.1s	1284	2.3s	1277
RejectionSampling	10.1s	1724	10.0s	1566	10.1s	1492	10.3s	1420	13.6s	1325	19.9s	1282	28.7s	1251
Qkmeans	6.5	1733	6.5	1568	14	1495	23	1423	80	1328	146	1283	220	1251

Table 17. Seeding benchmark on har. Time in ms, cost $\times 10^2$.

Algorithm	$k = 10$		$k = 50$		$k = 100$		$k = 200$		$k = 500$		$k = 750$		$k = 1000$	
	Time	Cost	Time	Cost	Time	Cost	Time	Cost	Time	Cost	Time	Cost	Time	Cost
AFKMC ²	2.7	2230	13	1587	40	1401	148	1230	931	1007	2.2s	907	4.2s	829
PRONECoreset	8.3	2408	29	1674	57	1440	147	1273	449	1049	617	949	905	880
FastKMeans	140	2305	138	1623	201	1483	147	1312	170	1094	313	996	247	917
RejectionSampling	913	2244	873	1596	895	1415	1.0s	1236	2.0s	1016	3.6s	911	6.1s	832
Qkmeans	7.5	2235	14	1594	42	1421	130	1245	595	1018	1.0s	916	1.6s	841

Table 18. Seeding benchmark on susy. Time in ms, cost $\times 10^6$.

Algorithm	$k = 10$		$k = 50$		$k = 100$		$k = 200$		$k = 500$		$k = 750$		$k = 1000$	
	Time	Cost	Time	Cost	Time	Cost	Time	Cost	Time	Cost	Time	Cost	Time	Cost
AFKMC ²	2.8	1.01	4.8	0.62	8.1	0.50	17	0.41	72	0.32	187	0.28	302	0.26
PRONECoreset	20	1.17	45	0.67	99	0.54	152	0.44	288	0.33	404	0.29	521	0.27
FastKMeans	1.1s	1.01	1.1s	0.63	1.1s	0.51	1.2s	0.43	1.3s	0.33	1.4s	0.29	1.5s	0.29
RejectionSampling	1.5s	0.99	1.5s	0.61	1.5s	0.50	1.6s	0.41	2.0s	0.32	2.9s	0.28	4.3s	0.26
Qkmeans	2.4	1.06	7.1	0.62	17	0.50	52	0.41	264	0.32	437	0.28	518	0.26